```
--------------------------------------------------------------------------
-- FILE : rgb_kmeans_cluster_engine.vhd
-- Author      : Sakinder
-- DESCRIPTION
-- This version contains extensive inline comments explaining the behavior
-- of the module for readability, maintainability, and documentation.
--
-- COMMENT COVERAGE
-- • Entity interface explanation
-- • Generics and configuration parameters
-- • Internal signal roles
-- • Sequential processes and clocking
-- • Pipeline data flow
-- • Computational stages
--
-- Compatible with standard IEEE VHDL tools.
--------------------------------------------------------------------------


--------------------------------------------------------------------------
-- Module : rgb_kmeans_cluster_engine
-- Description :
-- Refined version of original clustering module.
--
-- Function:
--   Performs RGB color clustering for real-time image processing.
--   Incoming pixel RGB values are compared with centroid lookup tables
--   and assigned to the nearest cluster.
--
-- Key Processing Steps:
--   1. Receive RGB pixel stream
--   2. Compare pixel against K cluster centroids
--   3. Compute distance / similarity metric
--   4. Select closest centroid
--   5. Output clustered RGB value
--
-- Notes:
--   Signal names simplified for readability.
--   Functional behavior unchanged.
--------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.fixed_pkg.all;
use work.float_pkg.all;
use work.constants_package.all;
use work.vpf_records.all;
use work.ports_package.all;
use work.vfp_pkg.all;
-- Entity declaration: defines the external interface (ports/generics) of the module
entity rgb_kmeans_cluster_engine is
-- Generic parameters: compile time configuration values
generic (
    i_data_width   : integer := 8);
-- Port definitions: input/output interface signals
port (
    clk            : in std_logic;
    rst_n          : in std_logic;
    pixel_in_rgb           : in channel;
    centroid_lut_select : in natural;
    centroid_lut_in       : in std_logic_vector(23 downto 0);
    centroid_lut_out      : out std_logic_vector(31 downto 0);
    k_ind_w        : in natural;
    k_ind_r        : in natural;
    pixel_out_rgb           : out channel);
-- End of design block
end rgb_kmeans_cluster_engine;
-- Architecture block: contains internal implementation of the entity
architecture arch of rgb_kmeans_cluster_engine is
-- Internal signal declaration: used for data transfer between concurrent blocks
    signal rgbSyncEol    : std_logic_vector(31 downto 0) := x"00000000";
-- Internal signal declaration: used for data transfer between concurrent blocks
    signal rgbSyncSof    : std_logic_vector(31 downto 0) := x"00000000";
-- Internal signal declaration: used for data transfer between concurrent blocks
```

```
75      signal rgbSyncEof    : std_logic_vector(31 downto 0) := x"00000000";
76   -- Internal signal declaration: used for data transfer between concurrent blocks
77      signal rgbSyncValid  : std_logic_vector(31 downto 0) := x"00000000";
78   -- Internal signal declaration: used for data transfer between concurrent blocks
79      signal k_rgb         : rgb_k_lut(0 to 30);
80      constant k_rgb_lut_1_l             : rgb_k_range(0 to 30) := (
81      (  0,  0,  0),
82      (255,240,230),
83      (240,220,210),
84      (240,220,170),
85      (240,210,150),
86      (240,210,180),
87      (240,200,140),
88      (240,200,160),
89      (240,180,150),
90      (240,150,140),
91      (180,140,160),
92      (180,160,140),
93      (180,170,120),
94      (180,145,100),
95      (180,140, 90),
96      (180,130, 80),
97      (150,140,130),
98      (150,120,110),
99      (150,100, 80),
100     (150, 80, 50),
101     (130,120,100),
102     (130,100, 80),
103     (130, 80, 60),
104     (130, 60, 30),
105     (120,110,100),
106     (120,100, 90),
107     (120, 80, 70),
108     (120, 60, 30),
109     (100, 90, 70),
110     (100, 70, 40),
111     (100, 50, 10));
112  -- Internal signal declaration: used for data transfer between concurrent blocks
113     signal k_rgb_lut_2_l             : rgb_k_range(0 to 30) := (
114     (  0,  0,  6),
115     ( 90, 90,  0),
116     ( 90, 50, 20),
117     ( 90, 40, 10),
118     ( 90, 40, 10),
119     ( 90, 30,  0),
120     ( 90, 30,  0),
121     ( 90, 20,  0),
122     ( 80, 80,  0),
123     ( 80, 50, 20),
124     ( 80, 40, 10),
125     ( 80, 40, 10),
126     ( 80, 30,  0),
127     ( 80, 30,  0),
128     ( 70, 70,  0),
129     ( 70, 50, 15),
130     ( 70, 30, 15),
131     ( 70, 10,  0),
132     ( 60, 60,  0),
133     ( 60, 30, 20),
134     ( 60, 20, 10),
135     ( 60, 10,  0),
136     ( 40, 40,  0),
137     ( 40, 20, 10),
138     ( 40, 10,  5),
139     ( 40, 10,  0),
140     ( 30, 30,  0),
141     ( 30, 20, 15),
142     ( 30, 15,  0),
143     ( 30, 10,  0),
144     ( 20, 10,  0));
145  -- Internal signal declaration: used for data transfer between concurrent blocks
146     signal k_rgb_lut_2ll             : rgb_k_range(0 to 30) := (
147     (  0,  0,  6),
148     ( 90, 90,  0),
```

```
149        ( 90, 50, 20),
150        ( 90, 40, 10),
151        ( 90, 40, 10),
152        ( 90, 30,  0),
153        ( 90, 30,  0),
154        ( 90, 20,  0),
155        ( 80, 80,  0),
156        ( 80, 50, 20),
157        ( 80, 40, 10),
158        ( 80, 40, 10),
159        ( 80, 30,  0),
160        ( 80, 30,  0),
161        ( 70, 70,  0),
162        ( 70, 50, 15),
163        ( 70, 30, 15),
164        ( 70, 10,  0),
165        ( 60, 60,  0),
166        ( 60, 30, 20),
167        ( 60, 20, 10),
168        ( 60, 10,  0),
169        ( 40, 40,  0),
170        ( 40, 20, 10),
171        ( 40, 10,  5),
172        ( 40, 10,  0),
173        ( 30, 30,  0),
174        ( 30, 20, 15),
175        ( 30, 15,  0),
176        ( 30, 10,  0),
177        ( 20, 10,  0));
178        constant k_rgb_lut_22l              : rgb_k_range(0 to 30) := (
179        (  0,  0,  6),
180        ( 90, 90,  0),
181        ( 90, 50, 20),
182        ( 90, 40, 10),
183        ( 90, 40, 10),
184        ( 90, 30,  0),
185        ( 90, 30,  0),
186        ( 90, 20,  0),
187        ( 80, 80,  0),
188        ( 80, 50, 20),
189        ( 80, 40, 10),
190        ( 80, 40, 10),
191        ( 80, 30,  0),
192        ( 80, 30,  0),
193        ( 70, 70,  0),
194        ( 70, 50, 15),
195        ( 70, 30, 15),
196        ( 70, 10,  0),
197        ( 60, 60,  0),
198        ( 60, 30, 20),
199        ( 60, 20, 10),
200        ( 60, 10,  0),
201        ( 40, 40,  0),
202        ( 40, 20, 10),
203        ( 40, 10,  5),
204        ( 40, 10,  0),
205        ( 30, 30,  0),
206        ( 30, 20, 15),
207        ( 30, 15,  0),
208        ( 30, 10,  0),
209        ( 20, 10,  0));
210        constant k_rgb_lut_0_l              : rgb_k_range(0 to 30) := (
211        ( 0,  0,  6),
212        (255,255,255),
213        (250,250,250),
214        (240,240,240),
215        (230,230,230),
216        (220,220,220),
217        (210,210,210),
218        (200,200,200),
219        (190,190,190),
220        (180,180,180),
221        (170,170,170),
222        (160,160,160),
```

```
223      (150,150,150),
224      (140,140,140),
225      (130,130,130),
226      (120,120,120),
227      (110,110,110),
228      (100,100,100),
229      ( 90, 90, 90),
230      ( 80, 80, 80),
231      ( 70, 70, 70),
232      ( 60, 60, 60),
233      ( 50, 50, 50),
234      ( 40, 40, 40),
235      ( 30, 30, 30),
236      ( 20, 20, 20),
237      ( 10, 10, 10),
238      (  0,  0,  0),
239      (255,255,255),
240      (150,150,150),
241      (100,100,100));
242  -- Internal signal declaration: used for data transfer between concurrent blocks
243      signal k_rgb_lut_4_l                : rgb_k_range(0 to 30) := (
244      (  0,  0,  0),
245      (240,200,  0),
246      (240,180,  0),
247      (230,180,  0),
248      (230,170,  0),
249      (230,160,  0),
250      (230,180,  0),
251      (200,160,  0),
252      (200,150,  0),
253      (200,140,  0),
254      (200,130,  0),
255      (200,100,  0),
256      (160,160,  0),
257      (160,120, 70),
258      (160,110, 50),
259      (160,100, 40),
260      (160, 90, 30),
261      (160, 50, 10),
262      (150, 80, 20),
263      (140,140,  0),
264      (140,100, 40),
265      (140, 80, 40),
266      (140, 60,  0),
267      (120,120,  0),
268      (120, 50,  0),
269      (120, 80,  0),
270      (120, 70,  0),
271      (100,100,  0),
272      (100, 50,  0),
273      (100, 90,  0),
274      (100, 25,  0));
275  -- Internal signal declaration: used for data transfer between concurrent blocks
276      signal k_rgb_lut_4ll                : rgb_k_range(0 to 30) := (
277      (  0,  0,  0),
278      (240,200,  0),
279      (240,180,  0),
280      (230,180,  0),
281      (230,170,  0),
282      (230,160,  0),
283      (230,180,  0),
284      (200,160,  0),
285      (200,150,  0),
286      (200,140,  0),
287      (200,130,  0),
288      (200,100,  0),
289      (160,160,  0),
290      (160,120, 70),
291      (160,110, 50),
292      (160,100, 40),
293      (160, 90, 30),
294      (160, 50, 10),
295      (150, 80, 20),
296      (140,140,  0),
```

```
297         (140,100, 40),
298         (140, 80, 40),
299         (140, 60,  0),
300         (120,120,  0),
301         (120, 50,  0),
302         (120, 80,  0),
303         (120, 70,  0),
304         (100,100,  0),
305         (100, 50,  0),
306         (100, 90,  0),
307         (100, 25,  0));
308     -- Internal signal declaration: used for data transfer between concurrent blocks
309         signal k_rgb_lut_3ll                : rgb_k_range(0 to 30) := (
310         (  0,  0,  0),
311         (240,200,  0),
312         (240,180,  0),
313         (230,180,  0),
314         (230,170,  0),
315         (230,160,  0),
316         (230,180,  0),
317         (200,160,  0),
318         (200,150,  0),
319         (200,140,  0),
320         (200,130,  0),
321         (200,100,  0),
322         (160,160,  0),
323         (160,120, 70),
324         (160,110, 50),
325         (160,100, 40),
326         (160, 90, 30),
327         (160, 50, 10),
328         (150, 80, 20),
329         (140,140,  0),
330         (140,100, 40),
331         (140, 80, 40),
332         (140, 60,  0),
333         (120,120,  0),
334         (120, 50,  0),
335         (120, 80,  0),
336         (120, 70,  0),
337         (100,100,  0),
338         (100, 50,  0),
339         (100, 90,  0),
340         (100, 25,  0));
341     -- Internal signal declaration: used for data transfer between concurrent blocks
342         signal k_rgb_lut_6ll                : rgb_k_range(0 to 30) := (
343         (  0,  0,  0),
344         (240,200,  0),
345         (240,180,  0),
346         (230,180,  0),
347         (230,170,  0),
348         (230,160,  0),
349         (230,180,  0),
350         (200,160,  0),
351         (200,150,  0),
352         (200,140,  0),
353         (200,130,  0),
354         (200,100,  0),
355         (160,160,  0),
356         (160,120, 70),
357         (160,110, 50),
358         (160,100, 40),
359         (160, 90, 30),
360         (160, 50, 10),
361         (150, 80, 20),
362         (140,140,  0),
363         (140,100, 40),
364         (140, 80, 40),
365         (140, 60,  0),
366         (120,120,  0),
367         (120, 50,  0),
368         (120, 80,  0),
369         (120, 70,  0),
370         (100,100,  0),
```

```
371      (100, 50,  0),
372      (100, 90,  0),
373      (100, 25,  0));
374  -- Internal signal declaration: used for data transfer between concurrent blocks
375      signal k_rgb_lut_3_l               : rgb_k_range(0 to 30) := (
376      (  0,  0,  0),
377      (240,200,  0),
378      (240,180,  0),
379      (230,180,  0),
380      (230,170,  0),
381      (230,160,  0),
382      (230,180,  0),
383      (200,160,  0),
384      (200,150,  0),
385      (200,140,  0),
386      (200,130,  0),
387      (200,100,  0),
388      (160,160,  0),
389      (160,120, 70),
390      (160,110, 50),
391      (160,100, 40),
392      (160, 90, 30),
393      (160, 50, 10),
394      (150, 80, 20),
395      (140,140,  0),
396      (140,100, 40),
397      (140, 80, 40),
398      (140, 60,  0),
399      (120,120,  0),
400      (120, 50,  0),
401      (120, 80,  0),
402      (120, 70,  0),
403      (100,100,  0),
404      (100, 50,  0),
405      (100, 90,  0),
406      (100, 25,  0));
407  -- Internal signal declaration: used for data transfer between concurrent blocks
408      signal k_rgb_lut_6_l               : rgb_k_range(0 to 30) := (
409      (  0,  0,  0),
410      (240,200,  0),
411      (240,180,  0),
412      (230,180,  0),
413      (230,170,  0),
414      (230,160,  0),
415      (230,180,  0),
416      (200,160,  0),
417      (200,150,  0),
418      (200,140,  0),
419      (200,130,  0),
420      (200,100,  0),
421      (160,160,  0),
422      (160,120, 70),
423      (160,110, 50),
424      (160,100, 40),
425      (160, 90, 30),
426      (160, 50, 10),
427      (150, 80, 20),
428      (140,140,  0),
429      (140,100, 40),
430      (140, 80, 40),
431      (140, 60,  0),
432      (120,120,  0),
433      (120, 50,  0),
434      (120, 80,  0),
435      (120, 70,  0),
436      (100,100,  0),
437      (100, 50,  0),
438      (100, 90,  0),
439      (100, 25,  0));
440
441      constant k_rgb_lut_41l             : rgb_k_range(0 to 30) := (
442      (  0,  0,  0),
443      (255,240,150),
444      (240,230,140),
```

```
445        (220,200,120),
446        (255,230, 75),
447        (240,220, 75),
448        (220,120, 75),
449        (200,180,158),
450        (180,160,128),
451        (160,140,118),
452        (140,120, 98),
453        (120,100, 88),
454        (100, 80, 78),
455        (255,210,128),
456        (240,190,120),
457        (220,170,110),
458        (200,150,100),
459        (180,130, 80),
460        (160,110, 60),
461        (140, 90, 40),
462        (120, 70, 20),
463        (100, 50,  0),
464        (255,150, 75),
465        (240,100, 75),
466        (220, 80, 75),
467        (200,140,100),
468        (180,120, 80),
469        (160,100, 60),
470        (140, 80, 40),
471        (120, 60, 20),
472        (100, 40,  0));
473        constant k_rgb_lut_42l              : rgb_k_range(0 to 30) := (
474        (  0,  0,  0),
475        (255,251,242),
476        (255,241,232),
477        (255,231,202),
478        (255,221,182),
479        (255,211,122),
480        (255,201,142),
481        (255,191,132),
482        (255,181,122),
483        (255,171,112),
484        (230,191,172),
485        (230,181,162),
486        (230, 81, 62),
487        (220,181,142),
488        (220,171,132),
489        (200, 51, 12),
490        (200,181, 92),
491        (180,161,132),
492        (180,151,122),
493        (170,151,112),
494        (170,141,102),
495        (150, 31, 22),
496        (150, 71, 42),
497        (130, 91, 62),
498        (130, 81, 52),
499        (120, 81, 42),
500        (120, 71, 32),
501        (100, 51, 22),
502        (100, 51, 22),
503        (100, 51, 22),
504        ( 40, 21, 12));
505        constant k_rgb_lut_43l              : rgb_k_range(0 to 30) := (
506        (  0,  0,  0),
507        (240,230,140),
508        (240,230,140),
509        (240,230,140),
510        (240,220, 75),
511        (240,220, 75),
512        (240,220, 75),
513        (200,180,158),
514        (180,160,128),
515        (180,160,128),
516        (180,160,128),
517        (120,100, 88),
518        (120,100, 88),
```

```
519        (240,190,120),
520        (240,190,120),
521        (240,190,120),
522        (200,150,100),
523        (200,150,100),
524        (160,110, 60),
525        (160,110, 60),
526        (120, 70, 20),
527        (120, 70, 20),
528        (255,240,230),
529        (246,216,192),
530        (236,192,145),
531        (207,150, 95),
532        (161,110, 75),
533        (127, 68, 34),
534        (140, 80, 40),
535        (120, 60, 20),
536        (100, 40,  0));
537        constant k_rgb_lut_44l              : rgb_k_range(0 to 30) := (
538        (  0,  0,  0),
539        (240,230,140),
540        (240,230,140),
541        (240,230,140),
542        (240,230,140),
543        (240,230,140),
544        (240,230,140),
545        (180,160,128),
546        (180,160,128),
547        (180,160,128),
548        (180,160,128),
549        (180,160,128),
550        (120,100, 88),
551        (120,100, 88),
552        (120,100, 88),
553        (120,100, 88),
554        (200,150,100),
555        (200,150,100),
556        (200,150,100),
557        (200,150,100),
558        (120, 70, 20),
559        (120, 70, 20),
560        (255,240,230),
561        (246,216,192),
562        (236,192,145),
563        (207,150, 95),
564        (161,110, 75),
565        (127, 68, 34),
566        (140, 80, 40),
567        (120, 60, 20),
568        (100, 40,  0));
569        constant k_rgb_lut_45l              : rgb_k_range(0 to 30) := (
570        (  0,  0,  0),
571        (255,240,150),
572        (220,200,120),
573        (220,200,120),
574        (220,200,120),
575        (220,200,120),
576        (220,120, 75),
577        (160,140,118),
578        (160,140,118),
579        (160,140,118),
580        (100, 80, 78),
581        (100, 80, 78),
582        (100, 80, 78),
583        (255,240,230),
584        (255,240,230),
585        (246,216,192),
586        (236,192,145),
587        (207,150, 95),
588        (161,110, 75),
589        (127, 68, 34),
590        (120, 70, 20),
591        (100, 50,  0),
592        (255,150, 75),
```

```
593         (240,100, 75),
594         (220, 80, 75),
595         (200,140,100),
596         (180,120, 80),
597         (160,100, 60),
598         (140, 80, 40),
599         (120, 60, 20),
600         (100, 40,  0));
601         constant k_rgb_lut_46l              : rgb_k_range(0 to 30) := (
602         (  0,  0,  0),
603         (255,240,  0),
604         (240,230,  0),
605         (220,200,  0),
606         (255,230,  0),
607         (255,240,  0),
608         (246,216,  0),
609         (236,192,  0),
610         (220,120,  0),
611         (207,150,  0),
612         (165,100,  0),
613         (127, 68,  0),
614         (209,168,  0),
615         (200,180,  0),
616         (180,160,  0),
617         (160,140,  0),
618         (140,120,  0),
619         (120,100,  0),
620         (100, 80,  0),
621         (240,190,  0),
622         (200,150,  0),
623         (180,130,  0),
624         (160,110,  0),
625         (140, 90,  0),
626         (240,100,  0),
627         (220, 80,  0),
628         (180,120,  0),
629         (160,100,  0),
630         (140, 80,  0),
631         (120, 60,  0),
632         (100, 40,  0));
633         constant k_rgb_lut_47l              : rgb_k_range(0 to 30) := (
634         (  0,  0,  0),
635         (255,251,242),
636         (255,241,232),
637         (255,231,202),
638         (255,221,182),
639         (209,168,138),
640         (255,201,142),
641         (255,191,132),
642         (255,181,122),
643         (255,240,230),
644         (255,240,230),
645         (246,216,192),
646         (236,192,145),
647         (220,120, 75),
648         (207,150, 95),
649         (200,150,100),
650         (127, 68, 34),
651         (209,168,138),
652         (180,151,122),
653         (170,151,112),
654         (170,141,102),
655         (200,150,100),
656         (150, 71, 42),
657         (130, 91, 62),
658         (130, 81, 52),
659         (120, 81, 42),
660         (120, 71, 32),
661         (100, 51, 22),
662         (100, 51, 22),
663         (100, 51, 22),
664         ( 40, 21, 12));
665         constant k_rgb_lut_48l              : rgb_k_range(0 to 30) := (
666         (  0,  0,  0),
```

```
667         (240,230,140),
668         (240,220, 75),
669         (240,220, 75),
670         (255,240,230),
671         (255,240,230),
672         (246,216,192),
673         (236,192,145),
674         (220,120, 75),
675         (207,150, 95),
676         (165,100, 80),
677         (127, 68, 34),
678         (209,168,138),
679         (240,190,120),
680         (240,190,120),
681         (240,190,120),
682         (200,150,100),
683         (200,150,100),
684         (160,110, 60),
685         (160,110, 60),
686         (120, 70, 20),
687         (120, 70, 20),
688         (255,240,230),
689         (246,216,192),
690         (236,192,145),
691         (207,150, 95),
692         (161,110, 75),
693         (127, 68, 34),
694         (140, 80, 40),
695         (120, 60, 20),
696         (100, 40,  0));
697         constant k_rgb_lut_49l             : rgb_k_range(0 to 30) := (
698         (  0,  0,  0),
699         (240,230,140),
700         (240,230,140),
701         (240,230,140),
702         (240,230,140),
703         (240,230,140),
704         (240,230,140),
705         (180,160,128),
706         (180,160,128),
707         (180,160,128),
708         (180,160,128),
709         (180,160,128),
710         (120,100, 88),
711         (120,100, 88),
712         (120,100, 88),
713         (120,100, 88),
714         (200,150,100),
715         (200,150,100),
716         (200,150,100),
717         (200,150,100),
718         (120, 70, 20),
719         (120, 70, 20),
720         (255,240,230),
721         (246,216,192),
722         (236,192,145),
723         (207,150, 95),
724         (161,110, 75),
725         (127, 68, 34),
726         (140, 80, 40),
727         (120, 60, 20),
728         (100, 40,  0));
729         constant k_rgb_lut_50l             : rgb_k_range(0 to 30) := (
730         (  0,  0,  0),
731         (255,240,150),
732         (220,200,120),
733         (220,200,120),
734         (220,200,120),
735         (209,168,138),
736         (220,120, 75),
737         (165,100, 80),
738         (165,100, 80),
739         (165,100, 80),
740         (100, 80, 78),
```

```
741        (100, 80, 78),
742        (100, 80, 78),
743        (255,240,230),
744        (255,240,230),
745        (246,216,192),
746        (236,192,145),
747        (207,150, 95),
748        (161,110, 75),
749        (127, 68, 34),
750        (120, 70, 20),
751        (100, 50,  0),
752        (255,150, 75),
753        (240,100, 75),
754        (220, 80, 75),
755        (200,140,100),
756        (180,120, 80),
757        (160,100, 60),
758        (140, 80, 40),
759        (120, 60, 20),
760        (100, 40,  0));
761        constant k_rgb_lut_51l              : rgb_k_range(0 to 30) := (
762        (  0,  0,  0),
763        (150,140,130),
764        (148,137,120),
765        (146,134,110),
766        (144,131,100),
767        (142,128, 90),
768        (140,125, 80),
769        (138,122, 70),
770        (136,119, 60),
771        (134,116, 50),
772        (132,113, 40),
773        (130,110, 30),
774        (128,107, 20),
775        (126,104, 10),
776        (124,101, 0),
777        (122, 98, 10),
778        (120, 95, 20),
779        (118, 92, 30),
780        (116, 89, 40),
781        (114, 86, 50),
782        (112, 83, 60),
783        (110, 80, 70),
784        (108, 77, 10),
785        (106, 74, 20),
786        (104, 71, 30),
787        (102, 68, 40),
788        (100, 65, 50),
789        (100, 62, 20),
790        (100, 59, 30),
791        (100, 56, 40),
792        (100, 53, 0));
793        constant k_rgb_lut_52l              : rgb_k_range(0 to 30) := (
794        (  0,  0,  0),
795        (240,200,150),
796        (200,160,120),
797        (170,110, 60),
798        (144,131,100),
799        (142,128, 90),
800        (140,125, 80),
801        (138,122, 70),
802        (136,119, 60),
803        (134,116, 50),
804        (132,113, 40),
805        (130,110, 30),
806        (128,107, 20),
807        (126,104, 10),
808        (124,101, 0),
809        (122, 98, 10),
810        (120, 95, 20),
811        (118, 92, 30),
812        (116, 89, 40),
813        (114, 86, 50),
814        (112, 83, 60),
```

```
815        (110, 80, 70),
816        (108, 77, 10),
817        (106, 74, 20),
818        (104, 71, 30),
819        (102, 68, 40),
820        (100, 65, 50),
821        (100, 62, 20),
822        (100, 59, 30),
823        (100, 56, 40),
824        (100, 53, 0));
825        constant k_rgb_lut_53l              : rgb_k_range(0 to 30) := (
826        (  0,  0,  0),
827        (240,200,150),
828        (200,160,120),
829        (170,110, 60),
830        (144,131,100),
831        (144,131,100),
832        (144,131,100),
833        (144,131,100),
834        (128,107, 20),
835        (128,107, 20),
836        (128,107, 20),
837        (128,107, 20),
838        (128,107, 20),
839        (112, 83, 60),
840        (112, 83, 60),
841        (112, 83, 60),
842        (112, 83, 60),
843        (112, 83, 60),
844        (112, 83, 60),
845        (112, 83, 60),
846        (112, 83, 60),
847        (110, 80, 70),
848        (108, 77, 10),
849        (106, 74, 20),
850        (104, 71, 30),
851        (102, 68, 40),
852        (100, 65, 50),
853        (100, 62, 20),
854        (100, 56, 40),
855        (100, 56, 40),
856        (100, 56, 40));
857        constant k_rgb_lut_54l              : rgb_k_range(0 to 30) := (
858        (  0,  0,  0),
859        (240,200,150),
860        (200,160,120),
861        (170,110, 60),
862        (144,131,100),
863        (144,131,100),
864        (144,131,100),
865        (144,131,100),
866        (128,107, 20),
867        (128,107, 20),
868        (128,107, 20),
869        (128,107, 20),
870        (128,107, 20),
871        (112, 83, 60),
872        (112, 83, 60),
873        (112, 83, 60),
874        (112, 83, 60),
875        (112, 83, 60),
876        (112, 83, 60),
877        (112, 83, 60),
878        (112, 83, 60),
879        (100, 56, 40),
880        (100, 56, 40),
881        (100, 56, 40),
882        (100, 56, 40),
883        (100, 56, 40),
884        (100, 56, 40),
885        (100, 56, 40),
886        (100, 56, 40),
887        (100, 56, 40),
888        (100, 56, 40));
```

```
889     constant k_rgb_lut_55l             : rgb_k_range(0 to 30) := (
890     (  0,  0,  0),
891     (240,200,150),
892     (144,131,100),
893     (144,131,100),
894     (144,131,100),
895     (144,131,100),
896     (144,131,100),
897     (144,131,100),
898     (128,107, 20),
899     (128,107, 20),
900     (128,107, 20),
901     (128,107, 20),
902     (128,107, 20),
903     (112, 83, 60),
904     (112, 83, 60),
905     (112, 83, 60),
906     (112, 83, 60),
907     (112, 83, 60),
908     (112, 83, 60),
909     (112, 83, 60),
910     (112, 83, 60),
911     (100, 56, 40),
912     (100, 56, 40),
913     (100, 56, 40),
914     (100, 56, 40),
915     (100, 56, 40),
916     (100, 56, 40),
917     (100, 56, 40),
918     (100, 56, 40),
919     (100, 56, 40),
920     (100, 56, 40));
921     constant k_rgb_lut_56l             : rgb_k_range(0 to 30) := (
922     (  0,  0,  0),
923     (240,200,  0),
924     (240,180,  0),
925     (230,180,  0),
926     (230,170,  0),
927     (230,160,  0),
928     (230,180,  0),
929     (200,160,  0),
930     (200,150,  0),
931     (200,140,  0),
932     (200,130,  0),
933     (200,200,  0),
934     (160,160,  0),
935     (140,140,  0),
936     (120,120,  0),
937     (100,100,  0),
938     (160,120, 70),
939     (160,110, 50),
940     (160,100, 40),
941     (160, 90, 30),
942     (160, 80, 20),
943     (160, 70, 10),
944     (140,100, 40),
945     (140,100, 30),
946     (140,100, 20),
947     (140,100, 10),
948     (120, 80,  0),
949     (120, 70,  0),
950     (100, 90,  0),
951     (100, 80,  0),
952     (100, 70,  0));
953     constant k_rgb_lut_57l             : rgb_k_range(0 to 30) := (
954     (  0,  0,  0),
955     (240,240,  0),
956     (240,200,  0),
957     (240,180,  0),
958     (230,230,  0),
959     (230,160,  0),
960     (230,180,  0),
961     (200,160,  0),
962     (200,150,  0),
```

```
963        (200,140,  0),
964        (200,130,  0),
965        (200,200,  0),
966        (160,160,  0),
967        (140,140,  0),
968        (120,120,  0),
969        (100,100,  0),
970        (160,120,  0),
971        (160,110,  0),
972        (160,100,  0),
973        (160, 90,  0),
974        (160, 80,  0),
975        (160, 70,  0),
976        (140,100,  0),
977        (140,100,  0),
978        (140,100,  0),
979        (140,100,  0),
980        (120, 80,  0),
981        (120, 70,  0),
982        (100, 90,  0),
983        (100, 80,  0),
984        (100, 70,  0));
985        constant k_rgb_lut_58l             : rgb_k_range(0 to 30) := (
986        (  0,  0,  0),
987        (240,240,  0),
988        (240,200,  0),
989        (240,180,  0),
990        (230,230,  0),
991        (230,160,  0),
992        (230,180,  0),
993        (255,240,230),-- required brown colors
994        (246,216,192),-- required brown colors
995        (236,192,145),-- required brown colors
996        (220,120, 75),-- required brown colors
997        (207,150, 95),-- required brown colors
998        (165,100, 80),-- required brown colors
999        (127, 68, 34),-- required brown colors
1000       (209,168,138),-- required brown colors
1001       (200,150,100),-- required brown colors
1002       (200,100,100),
1003       (200,200,  0),
1004       (160,160,  0),
1005       (120,120,  0),
1006       (100,100,  0),
1007       (160,120,  0),
1008       (160,110,  0),
1009       (160,100,  0),
1010       (160, 90,  0),
1011       (160, 80,  0),
1012       (140,100,  0),
1013       (100, 50, 50),
1014       (120, 80,  0),
1015       (100, 90,  0),
1016       (100, 80,  0));
1017       constant k_rgb_lut_59l             : rgb_k_range(0 to 30) := (
1018       (  0,  0,  0),
1019       (240,200,  0),
1020       (240,180,  0),
1021       (230,180,  0),
1022       (230,170,  0),
1023       (230,160,  0),
1024       (230,180,  0),
1025       (200,160,  0),
1026       (200,150,  0),
1027       (200,140,  0),
1028       (200,130,  0),
1029       (200,100,  0),
1030       (160,160,  0),
1031       (160,120, 70),
1032       (160,110, 50),
1033       (160,100, 40),
1034       (160, 90, 30),
1035       (160, 50, 10),
1036       (150, 80, 20),
```

```
1037      (140,140,  0),
1038      (140,100, 40),
1039      (140, 80, 40),
1040      (140, 60,  0),
1041      (120,120,  0),
1042      (120, 50,  0),
1043      (120, 80,  0),
1044      (120, 70,  0),
1045      (100,100,  0),
1046      (100, 50,  0),
1047      (100, 90,  0),
1048      (100, 25,  0));
1049
1050
1051
1052  -- Internal signal declaration: used for data transfer between concurrent blocks
1053      signal k1_rgb                 : rgb_k_lut(0 to 30);
1054  -- Internal signal declaration: used for data transfer between concurrent blocks
1055      signal k2_rgb                 : rgb_k_lut(0 to 30);
1056  -- Internal signal declaration: used for data transfer between concurrent blocks
1057      signal k3_rgb                 : rgb_k_lut(0 to 30);
1058  -- Internal signal declaration: used for data transfer between concurrent blocks
1059      signal k4_rgb                 : rgb_k_lut(0 to 30);
1060  -- Internal signal declaration: used for data transfer between concurrent blocks
1061      signal k5_rgb                 : rgb_k_lut(0 to 30);
1062  -- Internal signal declaration: used for data transfer between concurrent blocks
1063      signal k6_rgb                 : rgb_k_lut(0 to 30);
1064  -- Internal signal declaration: used for data transfer between concurrent blocks
1065      signal k7_rgb                 : rgb_k_lut(0 to 30);
1066  -- Internal signal declaration: used for data transfer between concurrent blocks
1067      signal k8_rgb                 : rgb_k_lut(0 to 30);
1068  -- Internal signal declaration: used for data transfer between concurrent blocks
1069      signal k9_rgb                 : rgb_k_lut(0 to 30);
1070  -- Internal signal declaration: used for data transfer between concurrent blocks
1071      signal k10rgb                 : rgb_k_lut(0 to 30);
1072  -- Internal signal declaration: used for data transfer between concurrent blocks
1073      signal k11rgb                 : rgb_k_lut(0 to 30);
1074  -- Internal signal declaration: used for data transfer between concurrent blocks
1075      signal k12rgb                 : rgb_k_lut(0 to 30);
1076  -- Internal signal declaration: used for data transfer between concurrent blocks
1077      signal k13rgb                 : rgb_k_lut(0 to 30);
1078  -- Internal signal declaration: used for data transfer between concurrent blocks
1079      signal k14rgb                 : rgb_k_lut(0 to 30);
1080  -- Internal signal declaration: used for data transfer between concurrent blocks
1081      signal k15rgb                 : rgb_k_lut(0 to 30);
1082  -- Internal signal declaration: used for data transfer between concurrent blocks
1083      signal k16rgb                 : rgb_k_lut(0 to 30);
1084  -- Internal signal declaration: used for data transfer between concurrent blocks
1085      signal k17rgb                 : rgb_k_lut(0 to 30);
1086  -- Internal signal declaration: used for data transfer between concurrent blocks
1087      signal k18rgb                 : rgb_k_lut(0 to 30);
1088  -- Internal signal declaration: used for data transfer between concurrent blocks
1089      signal k19rgb                 : rgb_k_lut(0 to 30);
1090  -- Internal signal declaration: used for data transfer between concurrent blocks
1091      signal k20rgb                 : rgb_k_lut(0 to 30);
1092  -- Internal signal declaration: used for data transfer between concurrent blocks
1093      signal k21rgb                 : rgb_k_lut(0 to 30);
1094  -- Internal signal declaration: used for data transfer between concurrent blocks
1095      signal k22rgb                 : rgb_k_lut(0 to 30);
1096  -- Internal signal declaration: used for data transfer between concurrent blocks
1097      signal k23rgb                 : rgb_k_lut(0 to 30);
1098  -- Internal signal declaration: used for data transfer between concurrent blocks
1099      signal k24rgb                 : rgb_k_lut(0 to 30);
1100  -- Internal signal declaration: used for data transfer between concurrent blocks
1101      signal k25rgb                 : rgb_k_lut(0 to 30);
1102  -- Internal signal declaration: used for data transfer between concurrent blocks
1103      signal k26rgb                 : rgb_k_lut(0 to 30);
1104  -- Internal signal declaration: used for data transfer between concurrent blocks
1105      signal k27rgb                 : rgb_k_lut(0 to 30);
1106  -- Internal signal declaration: used for data transfer between concurrent blocks
1107      signal rgb                    : k_val_rgb(0 to 30);
1108  -- Internal signal declaration: used for data transfer between concurrent blocks
1109      signal thr1                   : thr_record;
1110  -- Internal signal declaration: used for data transfer between concurrent blocks
```

```
1111      signal thr2                   : thr_record;
1112   -- Internal signal declaration: used for data transfer between concurrent blocks
1113      signal thr3                   : thr_record;
1114   -- Internal signal declaration: used for data transfer between concurrent blocks
1115      signal thr4                   : thr_record;
1116   -- Internal signal declaration: used for data transfer between concurrent blocks
1117      signal thr5                   : thr_record;
1118   -- Internal signal declaration: used for data transfer between concurrent blocks
1119      signal threshold              : integer;
1120   -- Internal signal declaration: used for data transfer between concurrent blocks
1121      signal threshold_lms_1        : integer;
1122   -- Internal signal declaration: used for data transfer between concurrent blocks
1123      signal threshold_lms_2        : integer;
1124   -- Internal signal declaration: used for data transfer between concurrent blocks
1125      signal threshold_lms_3        : integer;
1126   -- Internal signal declaration: used for data transfer between concurrent blocks
1127      signal threshold_lms_4        : integer;
1128   -- Internal signal declaration: used for data transfer between concurrent blocks
1129      signal threshold_lms_5        : integer;
1130   -- Internal signal declaration: used for data transfer between concurrent blocks
1131      signal threshold_lms_6        : integer;
1132   -- Internal signal declaration: used for data transfer between concurrent blocks
1133      signal threshold_lms_7        : integer;
1134   -- Internal signal declaration: used for data transfer between concurrent blocks
1135      signal threshold_lms_8        : integer;
1136   -- Internal signal declaration: used for data transfer between concurrent blocks
1137      signal threshold_lms_9        : integer;
1138   -- Internal signal declaration: used for data transfer between concurrent blocks
1139      signal threshold_lms10        : integer;
1140   -- Internal signal declaration: used for data transfer between concurrent blocks
1141      signal threshold_lms11        : integer;
1142   -- Internal signal declaration: used for data transfer between concurrent blocks
1143      signal threshold_lms12        : integer;
1144   -- Internal signal declaration: used for data transfer between concurrent blocks
1145      signal threshold_lms13        : integer;
1146   -- Internal signal declaration: used for data transfer between concurrent blocks
1147      signal threshold_lms14        : integer;
1148   -- Internal signal declaration: used for data transfer between concurrent blocks
1149      signal threshold_lms15        : integer;
1150   -- Internal signal declaration: used for data transfer between concurrent blocks
1151      signal threshold_lms16        : integer;
1152   -- Internal signal declaration: used for data transfer between concurrent blocks
1153      signal threshold_lms17        : integer;
1154   -- Internal signal declaration: used for data transfer between concurrent blocks
1155      signal threshold_lms18        : integer;
1156   -- Internal signal declaration: used for data transfer between concurrent blocks
1157      signal threshold_lms19        : integer;
1158   -- Internal signal declaration: used for data transfer between concurrent blocks
1159      signal threshold_lms20        : integer;
1160   -- Internal signal declaration: used for data transfer between concurrent blocks
1161      signal threshold_lms21        : integer;
1162   -- Internal signal declaration: used for data transfer between concurrent blocks
1163      signal threshold_red1         : integer;
1164   -- Internal signal declaration: used for data transfer between concurrent blocks
1165      signal threshold_red2         : integer;
1166   -- Internal signal declaration: used for data transfer between concurrent blocks
1167      signal threshold_gre1         : integer;
1168   -- Internal signal declaration: used for data transfer between concurrent blocks
1169      signal threshold_gre2         : integer;
1170   -- Internal signal declaration: used for data transfer between concurrent blocks
1171      signal threshold_blu1         : integer;
1172   -- Internal signal declaration: used for data transfer between concurrent blocks
1173      signal threshold_blu2         : integer;
1174   -- Internal signal declaration: used for data transfer between concurrent blocks
1175      signal threshold_lms          : integer;
1176   -- Internal signal declaration: used for data transfer between concurrent blocks
1177      signal rgb_min1               : integer;
1178   -- Internal signal declaration: used for data transfer between concurrent blocks
1179      signal rgb_max1               : integer;
1180   -- Internal signal declaration: used for data transfer between concurrent blocks
1181      signal rgb_max2               : integer;
1182   -- Internal signal declaration: used for data transfer between concurrent blocks
1183      signal rgb_max3               : integer;
1184   -- Internal signal declaration: used for data transfer between concurrent blocks
```

```
1185      signal rgb_max4              : integer;
1186  -- Internal signal declaration: used for data transfer between concurrent blocks
1187      signal rgb_max5              : integer;
1188  -- Internal signal declaration: used for data transfer between concurrent blocks
1189      signal rgb_max6              : integer;
1190  -- Internal signal declaration: used for data transfer between concurrent blocks
1191      signal rgb_max7              : integer;
1192  -- Internal signal declaration: used for data transfer between concurrent blocks
1193      signal rgb_max8              : integer;
1194  -- Internal signal declaration: used for data transfer between concurrent blocks
1195      signal rgb_max9              : integer;
1196  -- Internal signal declaration: used for data transfer between concurrent blocks
1197      signal rgb_max10             : integer;
1198  -- Internal signal declaration: used for data transfer between concurrent blocks
1199      signal rgb_max11             : integer;
1200  -- Internal signal declaration: used for data transfer between concurrent blocks
1201      signal rgb_max12             : integer;
1202  -- Internal signal declaration: used for data transfer between concurrent blocks
1203      signal rgb_max13             : integer;
1204  -- Internal signal declaration: used for data transfer between concurrent blocks
1205      signal rgb_max14             : integer;
1206  -- Internal signal declaration: used for data transfer between concurrent blocks
1207      signal rgb_max15             : integer;
1208  -- Internal signal declaration: used for data transfer between concurrent blocks
1209      signal rgb_max16             : integer;
1210  -- Internal signal declaration: used for data transfer between concurrent blocks
1211      signal rgb_max17             : integer;
1212  -- Internal signal declaration: used for data transfer between concurrent blocks
1213      signal rgb_max18             : integer;
1214  -- Internal signal declaration: used for data transfer between concurrent blocks
1215      signal rgb_max19             : integer;
1216  -- Internal signal declaration: used for data transfer between concurrent blocks
1217      signal rgb_max20             : integer;
1218  -- Internal signal declaration: used for data transfer between concurrent blocks
1219      signal rgb_max21             : integer;
1220  -- Internal signal declaration: used for data transfer between concurrent blocks
1221      signal rgb_max22             : integer;
1222  -- Internal signal declaration: used for data transfer between concurrent blocks
1223      signal rgb_std1sync          : channel;
1224  -- Internal signal declaration: used for data transfer between concurrent blocks
1225      signal rgb_std2sync          : channel;
1226  -- Internal signal declaration: used for data transfer between concurrent blocks
1227      signal rgb_std3sync          : channel;
1228  -- Internal signal declaration: used for data transfer between concurrent blocks
1229      signal rgb_std4sync          : channel;
1230  -- Internal signal declaration: used for data transfer between concurrent blocks
1231      signal rgb_sync1             : intChannel;
1232  -- Internal signal declaration: used for data transfer between concurrent blocks
1233      signal rgb_sync2             : intChannel;
1234  -- Internal signal declaration: used for data transfer between concurrent blocks
1235      signal rgb_sync3             : intChannel;
1236  -- Internal signal declaration: used for data transfer between concurrent blocks
1237      signal rgb_sync4             : intChannel;
1238  -- Internal signal declaration: used for data transfer between concurrent blocks
1239      signal rgb_sync5             : intChannel;
1240  -- Internal signal declaration: used for data transfer between concurrent blocks
1241      signal rgb_sync6             : intChannel;
1242  -- Internal signal declaration: used for data transfer between concurrent blocks
1243      signal rgb_sync7             : intChannel;
1244  -- Internal signal declaration: used for data transfer between concurrent blocks
1245      signal rgb_sync8             : intChannel;
1246  -- Internal signal declaration: used for data transfer between concurrent blocks
1247      signal rgb_sync9             : intChannel;
1248  -- Internal signal declaration: used for data transfer between concurrent blocks
1249      signal rgb_sync10            : intChannel;
1250  -- Internal signal declaration: used for data transfer between concurrent blocks
1251      signal rgb_sync11            : intChannel;
1252  -- Internal signal declaration: used for data transfer between concurrent blocks
1253      signal rgb_sync12            : intChannel;
1254  -- Internal signal declaration: used for data transfer between concurrent blocks
1255      signal rgb_sync13            : intChannel;
1256  -- Internal signal declaration: used for data transfer between concurrent blocks
1257      signal rgb_sync14            : intChannel;
1258  -- Internal signal declaration: used for data transfer between concurrent blocks
```

```
1259      signal rgb_sync15            : intChannel;
1260  -- Internal signal declaration: used for data transfer between concurrent blocks
1261      signal rgb_sync16            : intChannel;
1262  -- Internal signal declaration: used for data transfer between concurrent blocks
1263      signal rgb_sync17            : intChannel;
1264  -- Internal signal declaration: used for data transfer between concurrent blocks
1265      signal rgb_sync18            : intChannel;
1266  -- Internal signal declaration: used for data transfer between concurrent blocks
1267      signal rgb_sync19            : intChannel;
1268  -- Internal signal declaration: used for data transfer between concurrent blocks
1269      signal rgb_sync20            : intChannel;
1270  -- Internal signal declaration: used for data transfer between concurrent blocks
1271      signal rgb_sync21            : intChannel;
1272  -- Internal signal declaration: used for data transfer between concurrent blocks
1273      signal rgb_sync22            : intChannel;
1274  -- Internal signal declaration: used for data transfer between concurrent blocks
1275      signal rgb_sync23            : intChannel;
1276  -- Internal signal declaration: used for data transfer between concurrent blocks
1277      signal rgb_max               : integer;
1278  -- Internal signal declaration: used for data transfer between concurrent blocks
1279      signal rgb_mid               : integer;
1280  -- Internal signal declaration: used for data transfer between concurrent blocks
1281      signal rgb_min               : integer;
1282  -- Internal signal declaration: used for data transfer between concurrent blocks
1283      signal rgb_red               : std_logic_vector(7 downto 0);
1284  -- Internal signal declaration: used for data transfer between concurrent blocks
1285      signal rgb_gre               : std_logic_vector(7 downto 0);
1286  -- Internal signal declaration: used for data transfer between concurrent blocks
1287      signal rgb_blu               : std_logic_vector(7 downto 0);
1288      constant K_VALUE             : integer := 11;
1289      attribute keep : string;
1290      attribute keep of rgb_max : signal is "true";
1291      attribute keep of rgb_mid : signal is "true";
1292      attribute keep of rgb_min : signal is "true";
1293
1294  -- Begin implementation section
1295  begin
1296
1297  -- Sequential process block: executed on clock or sensitivity list events
1298  process (clk) begin
1299  -- Rising edge clock detection: ensures synchronous sequential logic
1300      if rising_edge(clk) then
1301          rgbSyncValid(0)  <= pixel_in_rgb.valid;
1302          for i in 0 to 30 loop
1303            rgbSyncValid(i+1)  <= rgbSyncValid(i);
1304  -- End of design block
1305          end loop;
1306  -- End of design block
1307      end if;
1308  -- End of design block
1309  end process;
1310
1311  -- Sequential process block: executed on clock or sensitivity list events
1312  process (clk) begin
1313  -- Rising edge clock detection: ensures synchronous sequential logic
1314      if rising_edge(clk) then
1315          rgbSyncEol(0)  <= pixel_in_rgb.eol;
1316          for i in 0 to 30 loop
1317            rgbSyncEol(i+1)  <= rgbSyncEol(i);
1318  -- End of design block
1319          end loop;
1320  -- End of design block
1321      end if;
1322  -- End of design block
1323  end process;
1324
1325  -- Sequential process block: executed on clock or sensitivity list events
1326  process (clk) begin
1327  -- Rising edge clock detection: ensures synchronous sequential logic
1328      if rising_edge(clk) then
1329          rgbSyncSof(0)  <= pixel_in_rgb.sof;
1330          for i in 0 to 30 loop
1331            rgbSyncSof(i+1)  <= rgbSyncSof(i);
1332  -- End of design block
```

```
1333            end loop;
1334    -- End of design block
1335        end if;
1336    -- End of design block
1337    end process;
1338
1339    -- Sequential process block: executed on clock or sensitivity list events
1340    process (clk) begin
1341    -- Rising edge clock detection: ensures synchronous sequential logic
1342        if rising_edge(clk) then
1343            rgbSyncEof(0)  <= pixel_in_rgb.eof;
1344            for i in 0 to 30 loop
1345              rgbSyncEof(i+1)  <= rgbSyncEof(i);
1346    -- End of design block
1347            end loop;
1348    -- End of design block
1349        end if;
1350    -- End of design block
1351    end process;
1352
1353    -- Sequential process block: executed on clock or sensitivity list events
1354    process (clk)begin
1355    -- Rising edge clock detection: ensures synchronous sequential logic
1356        if rising_edge(clk) then
1357            rgb_std1sync.red    <= pixel_in_rgb.red;
1358            rgb_std1sync.green  <= pixel_in_rgb.green;
1359            rgb_std1sync.blue   <= pixel_in_rgb.blue;
1360            rgb_std1sync.valid  <= pixel_in_rgb.valid;
1361    -- End of design block
1362        end if;
1363    -- End of design block
1364    end process;
1365
1366    -- Sequential process block: executed on clock or sensitivity list events
1367    process (clk)begin
1368    -- Rising edge clock detection: ensures synchronous sequential logic
1369        if rising_edge(clk) then
1370            rgb_std2sync    <= rgb_std1sync;
1371            rgb_std3sync    <= rgb_std2sync;
1372            rgb_std4sync    <= rgb_std3sync;
1373    -- End of design block
1374        end if;
1375    -- End of design block
1376    end process;
1377
1378    -- Sequential process block: executed on clock or sensitivity list events
1379    process (clk)begin
1380    -- Rising edge clock detection: ensures synchronous sequential logic
1381        if rising_edge(clk) then
1382                rgb_sync1.red    <= to_integer(unsigned(rgb_std1sync.red(9 downto 2)));
1383                rgb_sync1.green  <= to_integer(unsigned(rgb_std1sync.green(9 downto 2)));
1384                rgb_sync1.blue   <= to_integer(unsigned(rgb_std1sync.blue(9 downto 2)));
1385    -- End of design block
1386        end if;
1387    -- End of design block
1388    end process;
1389
1390    -- Sequential process block: executed on clock or sensitivity list events
1391    process (clk) begin
1392    -- Rising edge clock detection: ensures synchronous sequential logic
1393        if rising_edge(clk) then
1394            if ((rgb_sync1.red >= rgb_sync1.green) and (rgb_sync1.red >= rgb_sync1.blue)) then
1395                rgb_max1 <= rgb_sync1.red;
1396            elsif ((rgb_sync1.green >= rgb_sync1.red) and (rgb_sync1.green >= rgb_sync1.blue)) then
1397                rgb_max1 <= rgb_sync1.green;
1398            else
1399                rgb_max1 <= rgb_sync1.blue;
1400    -- End of design block
1401            end if;
1402    -- End of design block
1403        end if;
1404    -- End of design block
1405    end process;
1406
```

```
1407  -- Sequential process block: executed on clock or sensitivity list events
1408  process (clk) begin
1409  -- Rising edge clock detection: ensures synchronous sequential logic
1410      if rising_edge(clk) then
1411          if ((rgb_sync1.red <= rgb_sync1.green) and (rgb_sync1.red <= rgb_sync1.blue)) then
1412              rgb_min1 <= rgb_sync1.red;
1413          elsif((rgb_sync1.green <= rgb_sync1.red) and (rgb_sync1.green <= rgb_sync1.blue)) then
1414              rgb_min1 <= rgb_sync1.green;
1415          else
1416              rgb_min1 <= rgb_sync1.blue;
1417  -- End of design block
1418          end if;
1419  -- End of design block
1420      end if;
1421  -- End of design block
1422  end process;
1423
1424  -- Sequential process block: executed on clock or sensitivity list events
1425  process (clk) begin
1426  -- Rising edge clock detection: ensures synchronous sequential logic
1427      if rising_edge(clk) then
1428          if (rgb_sync2.red = rgb_max1) and (rgb_sync2.green = rgb_min1) then
1429              rgb_max <= rgb_sync2.red;
1430              rgb_mid <= rgb_sync2.blue;
1431              rgb_min <= rgb_sync2.green;
1432          elsif(rgb_sync2.red = rgb_max1) and (rgb_sync2.blue = rgb_min1)then
1433              rgb_max <= rgb_sync2.red;
1434              rgb_mid <= rgb_sync2.green;
1435              rgb_min <= rgb_sync2.blue;
1436          elsif(rgb_sync2.green = rgb_max1) and (rgb_sync2.blue = rgb_min1)then
1437              rgb_max <= rgb_sync2.green;
1438              rgb_mid <= rgb_sync2.red;
1439              rgb_min <= rgb_sync2.blue;
1440          elsif(rgb_sync2.green = rgb_max1) and (rgb_sync2.red = rgb_min1)then
1441              rgb_max <= rgb_sync2.green;
1442              rgb_mid <= rgb_sync2.blue;
1443              rgb_min <= rgb_sync2.red;
1444          elsif(rgb_sync2.blue = rgb_max1) and (rgb_sync2.red = rgb_min1)then
1445              rgb_max <= rgb_sync2.blue;
1446              rgb_mid <= rgb_sync2.green;
1447              rgb_min <= rgb_sync2.red;
1448          elsif(rgb_sync2.blue = rgb_max1) and (rgb_sync2.green = rgb_min1)then
1449              rgb_max <= rgb_sync2.blue;
1450              rgb_mid <= rgb_sync2.red;
1451              rgb_min <= rgb_sync2.green;
1452  -- End of design block
1453          end if;
1454  -- End of design block
1455      end if;
1456  -- End of design block
1457  end process;
1458
1459  -- Sequential process block: executed on clock or sensitivity list events
1460  process (clk)begin
1461  -- Rising edge clock detection: ensures synchronous sequential logic
1462      if rising_edge(clk) then
1463        rgb_max2        <= rgb_max1;
1464        rgb_max3        <= rgb_max2;
1465        rgb_max4        <= rgb_max3;
1466        rgb_max5        <= rgb_max4;
1467        rgb_max6        <= rgb_max5;
1468        rgb_max7        <= rgb_max6;
1469        rgb_max8        <= rgb_max7;
1470        rgb_max9        <= rgb_max8;
1471        rgb_max10       <= rgb_max9;
1472        rgb_max11       <= rgb_max10;
1473        rgb_max12       <= rgb_max11;
1474        rgb_max13       <= rgb_max12;
1475        rgb_max14       <= rgb_max13;
1476        rgb_max15       <= rgb_max14;
1477        rgb_max16       <= rgb_max15;
1478        rgb_max17       <= rgb_max16;
1479        rgb_max18       <= rgb_max17;
1480        rgb_max19       <= rgb_max18;
```

```
1481        rgb_max20       <= rgb_max19;
1482        rgb_max21       <= rgb_max20;
1483        rgb_max22       <= rgb_max21;
1484    -- End of design block
1485        end if;
1486    -- End of design block
1487    end process;
1488    -- Sequential process block: executed on clock or sensitivity list events
1489    process (clk)begin
1490    -- Rising edge clock detection: ensures synchronous sequential logic
1491        if rising_edge(clk) then
1492        rgb_sync2        <= rgb_sync1;
1493        rgb_sync3        <= rgb_sync2;
1494        rgb_sync4        <= rgb_sync3;
1495        rgb_sync5        <= rgb_sync4;
1496        rgb_sync6        <= rgb_sync5;
1497        rgb_sync7        <= rgb_sync6;
1498        rgb_sync8        <= rgb_sync7;
1499        rgb_sync9        <= rgb_sync8;
1500        rgb_sync10       <= rgb_sync9;
1501        rgb_sync11       <= rgb_sync10;
1502        rgb_sync12       <= rgb_sync11;
1503        rgb_sync13       <= rgb_sync12;
1504        rgb_sync14       <= rgb_sync13;
1505        rgb_sync15       <= rgb_sync14;
1506        rgb_sync16       <= rgb_sync15;
1507        rgb_sync17       <= rgb_sync16;
1508        rgb_sync18       <= rgb_sync17;
1509        rgb_sync19       <= rgb_sync18;
1510        rgb_sync20       <= rgb_sync19;
1511        rgb_sync21       <= rgb_sync20;
1512        rgb_sync22       <= rgb_sync21;
1513        rgb_sync23       <= rgb_sync22;
1514    -- End of design block
1515        end if;
1516    -- End of design block
1517    end process;
1518    -- best select is 18/17/16
1519    -- Sequential process block: executed on clock or sensitivity list events
1520    process (clk)begin
1521    -- Rising edge clock detection: ensures synchronous sequential logic
1522        if rising_edge(clk) then
1523            if (k_ind_w <= 30)then
1524                k_rgb_lut_4ll(k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16))));
                     k_rgb_lut_4ll(k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15 downto 8))));
                       k_rgb_lut_4ll(k_ind_w).min <= to_integer((unsigned(centroid_lut_in(7 downto 0))));
1525    -- End of design block
1526            end if;
1527    -- End of design block
1528        end if;
1529    -- End of design block
1530    end process;
1531
1532    -- Sequential process block: executed on clock or sensitivity list events
1533    process (clk)begin
1534    -- Rising edge clock detection: ensures synchronous sequential logic
1535        if rising_edge(clk) then
1536            if (k_ind_w >= 31 and k_ind_w <= 60) then
1537                k_rgb_lut_2ll(61-k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16
                    )))); k_rgb_lut_2ll(61-k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15 downto
                     8))));   k_rgb_lut_2ll(61-k_ind_w).min <= to_integer((unsigned(centroid_lut_in(7
                    downto 0))));
1538    -- End of design block
1539            end if;
1540    -- End of design block
1541        end if;
1542    -- End of design block
1543    end process;
1544
1545    -- Sequential process block: executed on clock or sensitivity list events
1546    process (clk)begin
1547    -- Rising edge clock detection: ensures synchronous sequential logic
1548        if rising_edge(clk) then
1549            if (k_ind_w >= 61 and k_ind_w <= 90) then
```

```
1550                k_rgb_lut_3ll(91-k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16
                )))); k_rgb_lut_3ll(91-k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15 downto
                 8))));   k_rgb_lut_3ll(91-k_ind_w).min <= to_integer((unsigned(centroid_lut_in(7
                downto 0))));
1551    -- End of design block
1552            end if;
1553    -- End of design block
1554        end if;
1555    -- End of design block
1556    end process;
1557
1558    -- Sequential process block: executed on clock or sensitivity list events
1559    process (clk)begin
1560    -- Rising edge clock detection: ensures synchronous sequential logic
1561        if rising_edge(clk) then
1562            if (k_ind_w >= 91 and k_ind_w <= 120) then
1563                k_rgb_lut_6ll(121-k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16
                )))); k_rgb_lut_6ll(121-k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15
                downto 8))));   k_rgb_lut_6ll(121-k_ind_w).min <= to_integer((unsigned(
                centroid_lut_in(7 downto 0))));
1564    -- End of design block
1565            end if;
1566    -- End of design block
1567        end if;
1568    -- End of design block
1569    end process;
1570
1571    -- Sequential process block: executed on clock or sensitivity list events
1572    process(clk)begin
1573    -- Rising edge clock detection: ensures synchronous sequential logic
1574        if rising_edge(clk) then
1575            if (k_ind_r <= 30)then
1576                centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(
                k_rgb_lut_4_l(k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_4_l(k_ind_r
                ).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_4_l(k_ind_r).min, 8));
1577            elsif(k_ind_w >= 31 and k_ind_w <= 60)then
1578                centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(
                k_rgb_lut_2_l(61-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_2_l(61-
                k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_2_l(61-k_ind_r).min, 8));
1579            elsif(k_ind_w >= 61 and k_ind_w <= 90)then
1580                centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(
                k_rgb_lut_3_l(91-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-
                k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).min, 8));
1581            elsif(k_ind_w >= 91 and k_ind_w <= 120)then
1582                centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(
                k_rgb_lut_6_l(121-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_6_l(121-
                k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_6_l(121-k_ind_r).min, 8));
1583            else
1584                centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(
                k_rgb_lut_3_l(91-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-
                k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).min, 8));
1585    -- End of design block
1586            end if;
1587    -- End of design block
1588        end if;
1589    -- End of design block
1590    end process;
1591
1592
1593
1594    -- Sequential process block: executed on clock or sensitivity list events
1595    process (clk)begin
1596    -- Rising edge clock detection: ensures synchronous sequential logic
1597        if rising_edge(clk) then
1598            if (k_ind_w <= 90)then
1599                k_rgb_lut_4_l <= k_rgb_lut_4ll;
1600                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1601                k_rgb_lut_2_l <= k_rgb_lut_2ll;
1602                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1603            elsif(k_ind_w = 201)then
1604                k_rgb_lut_4_l <= k_rgb_lut_42l;
1605                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1606                k_rgb_lut_2_l <= k_rgb_lut_22l;
1607                k_rgb_lut_6_l <= k_rgb_lut_6ll;
```

```
1608            elsif(k_ind_w = 202)then
1609                k_rgb_lut_4_l <= k_rgb_lut_43l;
1610                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1611                k_rgb_lut_2_l <= k_rgb_lut_22l;
1612                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1613            elsif(k_ind_w = 203)then
1614                k_rgb_lut_4_l <= k_rgb_lut_41l;
1615                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1616                k_rgb_lut_2_l <= k_rgb_lut_22l;
1617                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1618            elsif(k_ind_w = 204)then
1619                k_rgb_lut_4_l <= k_rgb_lut_42l;
1620                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1621                k_rgb_lut_2_l <= k_rgb_lut_22l;
1622                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1623            elsif(k_ind_w = 205)then
1624                k_rgb_lut_4_l <= k_rgb_lut_43l;
1625                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1626                k_rgb_lut_2_l <= k_rgb_lut_22l;
1627                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1628            elsif(k_ind_w = 206)then
1629                k_rgb_lut_4_l <= k_rgb_lut_44l;
1630                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1631                k_rgb_lut_2_l <= k_rgb_lut_22l;
1632                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1633            elsif(k_ind_w = 207)then
1634                k_rgb_lut_4_l <= k_rgb_lut_45l;
1635                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1636                k_rgb_lut_2_l <= k_rgb_lut_22l;
1637                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1638            elsif(k_ind_w = 208)then
1639                k_rgb_lut_4_l <= k_rgb_lut_46l;
1640                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1641                k_rgb_lut_2_l <= k_rgb_lut_22l;
1642                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1643            elsif(k_ind_w = 209)then
1644                k_rgb_lut_4_l <= k_rgb_lut_47l;
1645                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1646                k_rgb_lut_2_l <= k_rgb_lut_22l;
1647                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1648            elsif(k_ind_w =210)then
1649                k_rgb_lut_4_l <= k_rgb_lut_48l;
1650                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1651                k_rgb_lut_2_l <= k_rgb_lut_22l;
1652                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1653            elsif(k_ind_w =211)then
1654                k_rgb_lut_4_l <= k_rgb_lut_49l;
1655                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1656                k_rgb_lut_2_l <= k_rgb_lut_22l;
1657                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1658            elsif(k_ind_w =212)then
1659                k_rgb_lut_4_l <= k_rgb_lut_50l;
1660                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1661                k_rgb_lut_2_l <= k_rgb_lut_22l;
1662                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1663            elsif(k_ind_w =213)then
1664                k_rgb_lut_4_l <= k_rgb_lut_51l;
1665                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1666                k_rgb_lut_2_l <= k_rgb_lut_22l;
1667                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1668            elsif(k_ind_w =214)then
1669                k_rgb_lut_4_l <= k_rgb_lut_52l;
1670                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1671                k_rgb_lut_2_l <= k_rgb_lut_22l;
1672                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1673            elsif(k_ind_w =215)then
1674                k_rgb_lut_4_l <= k_rgb_lut_53l;
1675                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1676                k_rgb_lut_2_l <= k_rgb_lut_22l;
1677                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1678            elsif(k_ind_w =216)then
1679                k_rgb_lut_4_l <= k_rgb_lut_54l;
1680                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1681                k_rgb_lut_2_l <= k_rgb_lut_22l;
```

```
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w =217)then
            k_rgb_lut_4_l <= k_rgb_lut_55l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w =218)then
            k_rgb_lut_4_l <= k_rgb_lut_56l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w =219)then
            k_rgb_lut_4_l <= k_rgb_lut_57l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w =220)then
            k_rgb_lut_4_l <= k_rgb_lut_58l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w =221)then
            k_rgb_lut_4_l <= k_rgb_lut_59l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
-- End of design block
        end if;
-- End of design block
    end if;
-- End of design block
end process;
-- best select is 0
-- Sequential process block: executed on clock or sensitivity list events
process (clk)begin
-- Rising edge clock detection: ensures synchronous sequential logic
    if rising_edge(clk) then
        if (centroid_lut_select   = 0)then

            ------------------------------------------------------------------------------------------------------------------------
            if ((abs(rgb_sync2.red - rgb_sync2.green) <= 6) and (abs(rgb_sync2.red - rgb_sync2.blue) <= 6)) or (abs(rgb_sync2.red - rgb_sync2.blue) <= 6 and (abs(rgb_sync2.red - rgb_sync2.green) <= 6)) or (abs(rgb_sync2.green - rgb_sync2.blue) <= 6  and (abs(rgb_sync2.red - rgb_sync2.blue) <= 6))then
              for i in 0 to 30 loop
                k_rgb(i).red <=   k_rgb_lut_0_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_0_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_0_l(i).mid;
-- End of design block
              end loop;
            elsif (rgb_sync2.red = rgb_max1) then
                if (rgb_max1 - rgb_min1 >= 100) then
                    if (rgb_sync2.green = rgb_min1) then
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_1_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_1_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).mid;
-- End of design block
                      end loop;
                    else
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_1_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_1_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).min;
-- End of design block
                      end loop;
-- End of design block
                    end if;
                elsif (rgb_sync2.red >= 100) then
                    if (rgb_sync2.green = rgb_min1) then
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
-- End of design block
                      end loop;
                    else
```

```
1747                        for i in 0 to 30 loop
1748                          k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_4_l(
                              i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1749   -- End of design block
1750                        end loop;
1751   -- End of design block
1752                    end if;
1753                else
1754                    if (rgb_sync2.green = rgb_min1) then
1755                          for i in 0 to 30 loop
1756                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1757   -- End of design block
1758                          end loop;
1759                    else
1760                          for i in 0 to 30 loop
1761                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1762   -- End of design block
1763                          end loop;
1764   -- End of design block
1765                    end if;
1766   -- End of design block
1767                end if;
1768            elsif (rgb_sync2.green = rgb_max1) then
1769                if (rgb_max1 - rgb_min1 >= 100) then
1770                    if (rgb_sync2.red = rgb_min1) then
1771                          for i in 0 to 30 loop
1772                            k_rgb(i).red <=   k_rgb_lut_1_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_1_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).mid;
1773   -- End of design block
1774                          end loop;
1775                    else
1776                          for i in 0 to 30 loop
1777                            k_rgb(i).red <=   k_rgb_lut_1_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_1_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).min;
1778   -- End of design block
1779                          end loop;
1780   -- End of design block
1781                    end if;
1782                elsif(rgb_sync2.green >= 100) then
1783                    if (rgb_sync2.red = rgb_min1) then
1784                          for i in 0 to 30 loop
1785                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1786   -- End of design block
1787                          end loop;
1788                    else
1789                          for i in 0 to 30 loop
1790                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1791   -- End of design block
1792                          end loop;
1793   -- End of design block
1794                    end if;
1795                else
1796                    if (rgb_sync2.red = rgb_min1) then
1797                          for i in 0 to 30 loop
1798                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1799   -- End of design block
1800                          end loop;
1801                    else
1802                          for i in 0 to 30 loop
1803                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1804   -- End of design block
1805                          end loop;
1806   -- End of design block
1807                    end if;
1808   -- End of design block
1809                end if;
1810            else
1811                if (rgb_max1 - rgb_min1 >= 100) then
```

```
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_1_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_1_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).max;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_1_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_1_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).max;
-- End of design block
                          end loop;
-- End of design block
                    end if;
                elsif(rgb_sync2.blue >= 100) then
                        if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                          end loop;
                        else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                          end loop;
-- End of design block
                        end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                          end loop;
-- End of design block
                    end if;
-- End of design block
                end if;
-- End of design block
            end if;

            ------------------
```

```
1871                        for i in 0 to 30 loop
1872                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1873   -- End of design block
1874                        end loop;
1875   -- End of design block
1876                    end if;
1877                else
1878                    if (rgb_sync2.green = rgb_min1) then
1879                        for i in 0 to 30 loop
1880                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1881   -- End of design block
1882                        end loop;
1883                    else
1884                        for i in 0 to 30 loop
1885                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1886   -- End of design block
1887                        end loop;
1888   -- End of design block
1889                    end if;
1890   -- End of design block
1891                end if;
1892            -- GREEN MAX
1893            elsif (rgb_sync2.green = rgb_max1) then
1894                if(rgb_sync2.green >= 100) then
1895                    if (rgb_sync2.red = rgb_min1) then
1896                          for i in 0 to 30 loop
1897                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1898   -- End of design block
1899                          end loop;
1900                    else
1901                          for i in 0 to 30 loop
1902                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1903   -- End of design block
1904                          end loop;
1905   -- End of design block
1906                    end if;
1907                else
1908                    if (rgb_sync2.red = rgb_min1) then
1909                          for i in 0 to 30 loop
1910                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1911   -- End of design block
1912                          end loop;
1913                    else
1914                          for i in 0 to 30 loop
1915                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1916   -- End of design block
1917                          end loop;
1918   -- End of design block
1919                    end if;
1920   -- End of design block
1921                end if;
1922            else
1923            -- BLUE MAX
1924                if(rgb_sync2.blue >= 100) then
1925                    if (rgb_sync2.red = rgb_min1) then
1926                        for i in 0 to 30 loop
1927                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1928   -- End of design block
1929                        end loop;
1930                    else
1931                        for i in 0 to 30 loop
1932                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1933   -- End of design block
1934                        end loop;
1935   -- End of design block
```

```
1936                    end if;
1937                else
1938                    if (rgb_sync2.red = rgb_min1) then
1939                          for i in 0 to 30 loop
1940                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1941  -- End of design block
1942                          end loop;
1943                    else
1944                          for i in 0 to 30 loop
1945                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1946  -- End of design block
1947                          end loop;
1948  -- End of design block
1949                    end if;
1950  -- End of design block
1951                end if;
1952  -- End of design block
1953            end if;
1954
                ------------------------------------------------------------------------------------
                --------------------
1955        elsif(centroid_lut_select = 2) then
1956
                ------------------------------------------------------------------------------------
                --------------------
1957            if (rgb_sync2.red = rgb_max1) then
1958                if (rgb_sync2.red >= 100) then
1959                    if (rgb_sync2.green = rgb_min1) then
1960                        for i in 0 to 30 loop
1961                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1962  -- End of design block
1963                        end loop;
1964                    else
1965                        for i in 0 to 30 loop
1966                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1967  -- End of design block
1968                        end loop;
1969  -- End of design block
1970                    end if;
1971                else
1972                    if (rgb_sync2.green = rgb_min1) then
1973                        for i in 0 to 30 loop
1974                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1975  -- End of design block
1976                        end loop;
1977                    else
1978                        for i in 0 to 30 loop
1979                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1980  -- End of design block
1981                        end loop;
1982  -- End of design block
1983                    end if;
1984  -- End of design block
1985                end if;
1986            -- GREEN MAX
1987            elsif (rgb_sync2.green = rgb_max1) then
1988                if(rgb_sync2.green >= 100) then
1989                    if (rgb_sync2.red = rgb_min1) then
1990                          for i in 0 to 30 loop
1991                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1992  -- End of design block
1993                          end loop;
1994                    else
1995                          for i in 0 to 30 loop
1996                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1997  -- End of design block
```

```
                            end loop;
-- End of design block
                        end if;
                    else
                        if (rgb_sync2.red = rgb_min1) then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
-- End of design block
                            end loop;
-- End of design block
                        end if;
-- End of design block
                    end if;
                else
                -- BLUE MAX
                    if(rgb_sync2.blue >= 100) then
                        if (rgb_sync2.red = rgb_min1) then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                            end loop;
-- End of design block
                        end if;
                    else
                        if (rgb_sync2.red = rgb_min1) then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                            end loop;
-- End of design block
                        end if;
-- End of design block
                    end if;
-- End of design block
                end if;

                ------------------------------------------------------------------------------------------------------------------
            elsif(centroid_lut_select = 3) then

                ------------------------------------------------------------------------------------------------------------------
                if (rgb_sync2.red = rgb_max1) then
                    if (rgb_sync2.red >= 100) then
                        if (rgb_sync2.green = rgb_min1) then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
-- End of design block
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
```

```
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2061    -- End of design block
2062                            end loop;
2063    -- End of design block
2064                        end if;
2065                    else
2066                        if (rgb_sync2.green = rgb_min1) then
2067                            for i in 0 to 30 loop
2068                                k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2069    -- End of design block
2070                            end loop;
2071                        else
2072                            for i in 0 to 30 loop
2073                                k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2074    -- End of design block
2075                            end loop;
2076    -- End of design block
2077                        end if;
2078    -- End of design block
2079                    end if;
2080                -- GREEN MAX
2081                elsif (rgb_sync2.green = rgb_max1) then
2082                    if(rgb_sync2.green >= 100) then
2083                        if (rgb_sync2.red = rgb_min1) then
2084                              for i in 0 to 30 loop
2085                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2086    -- End of design block
2087                              end loop;
2088                        else
2089                              for i in 0 to 30 loop
2090                                k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
2091    -- End of design block
2092                              end loop;
2093    -- End of design block
2094                        end if;
2095                    else
2096                        if (rgb_sync2.red = rgb_min1) then
2097                              for i in 0 to 30 loop
2098                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2099    -- End of design block
2100                              end loop;
2101                        else
2102                              for i in 0 to 30 loop
2103                                k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
2104    -- End of design block
2105                              end loop;
2106    -- End of design block
2107                        end if;
2108    -- End of design block
2109                    end if;
2110                else
2111                -- BLUE MAX
2112                    if(rgb_sync2.blue >= 100) then
2113                        if (rgb_sync2.red = rgb_min1) then
2114                            for i in 0 to 30 loop
2115                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
2116    -- End of design block
2117                            end loop;
2118                        else
2119                            for i in 0 to 30 loop
2120                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
2121    -- End of design block
2122                            end loop;
2123    -- End of design block
2124                        end if;
2125                    else
```

```
2126                    if (rgb_sync2.red = rgb_min1) then
2127                          for i in 0 to 30 loop
2128                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
2129  -- End of design block
2130                          end loop;
2131                    else
2132                          for i in 0 to 30 loop
2133                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
2134  -- End of design block
2135                          end loop;
2136  -- End of design block
2137                    end if;
2138  -- End of design block
2139                end if;
2140  -- End of design block
2141            end if;
2142
                ------------------------------------------------------------------------------------
                --------------------
2143        elsif(centroid_lut_select = 4) then
2144
                ------------------------------------------------------------------------------------
                -------------------
2145            if (rgb_sync2.red = rgb_max1) then
2146                if (rgb_sync2.red >= 100) then
2147                    if (rgb_sync2.green = rgb_min1) then
2148                        for i in 0 to 30 loop
2149                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2150  -- End of design block
2151                        end loop;
2152                    else
2153                        for i in 0 to 30 loop
2154                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2155  -- End of design block
2156                        end loop;
2157  -- End of design block
2158                    end if;
2159                else
2160                    if (rgb_sync2.green = rgb_min1) then
2161                        for i in 0 to 30 loop
2162                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2163  -- End of design block
2164                        end loop;
2165                    else
2166                        for i in 0 to 30 loop
2167                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2168  -- End of design block
2169                        end loop;
2170  -- End of design block
2171                    end if;
2172  -- End of design block
2173                end if;
2174            -- GREEN MAX
2175            elsif (rgb_sync2.green = rgb_max1) then
2176                if(rgb_sync2.green >= 100) then
2177                    if (rgb_sync2.red = rgb_min1) then
2178                          for i in 0 to 30 loop
2179                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2180  -- End of design block
2181                          end loop;
2182                    else
2183                          for i in 0 to 30 loop
2184                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
2185  -- End of design block
2186                          end loop;
2187  -- End of design block
```

```
                    end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
-- End of design block
                          end loop;
-- End of design block
                    end if;
-- End of design block
                end if;
            else
            -- BLUE MAX
                if(rgb_sync2.blue >= 100) then
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                        end loop;
-- End of design block
                    end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                          end loop;
-- End of design block
                    end if;
-- End of design block
                end if;
-- End of design block
            end if;

            ------------------
```

```
2249                            k_rgb(i).red <=   k_rgb_lut_6_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_6_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).mid;
2250   -- End of design block
2251                        end loop;
2252   -- End of design block
2253                    end if;
2254                elsif(rgb_sync2.red >= 170) then
2255
                        ------------------------------------------------------------------------------
                        --------------------
2256                    if (rgb_sync2.green = rgb_min1) then
2257                        for i in 0 to 30 loop
2258                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
2259   -- End of design block
2260                        end loop;
2261                    else
2262                        for i in 0 to 30 loop
2263                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
2264   -- End of design block
2265                        end loop;
2266   -- End of design block
2267                    end if;
2268
                        ------------------------------------------------------------------------------
                        --------------------
2269                elsif(rgb_sync2.red >= 85) then
2270
                        ------------------------------------------------------------------------------
                        --------------------
2271                    if (rgb_sync2.green = rgb_min1) then
2272                        for i in 0 to 30 loop
2273                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2274   -- End of design block
2275                        end loop;
2276                    else
2277                        for i in 0 to 30 loop
2278                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2279   -- End of design block
2280                        end loop;
2281   -- End of design block
2282                    end if;
2283
                        ------------------------------------------------------------------------------
                        --------------------
2284                else
2285
                        ------------------------------------------------------------------------------
                        --------------------
2286                    if (rgb_sync2.green = rgb_min1) then
2287                        for i in 0 to 30 loop
2288                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2289   -- End of design block
2290                        end loop;
2291                    else
2292                        for i in 0 to 30 loop
2293                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2294   -- End of design block
2295                        end loop;
2296   -- End of design block
2297                    end if;
2298
                        ------------------------------------------------------------------------------
                        --------------------
2299   -- End of design block
2300                end if;
2301            -- GREEN MAX
2302            elsif (rgb_sync2.green = rgb_max1) then
2303                if(rgb_sync2.green >= 210) then
```

```
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_6_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).mid;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_6_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).mid;
-- End of design block
                          end loop;
-- End of design block
                    end if;
                elsif(rgb_sync2.green >= 170) then
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
-- End of design block
                          end loop;
-- End of design block
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                elsif(rgb_sync2.red >= 85) then

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
-- End of design block
                          end loop;
-- End of design block
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                else

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                          end loop;
-- End of design block
                    end if;
-- End of design block
                end if;
            else
```

```
2362                -- BLUE MAX
2363                    if(rgb_sync2.blue >= 210) then
2364                        if (rgb_sync2.red = rgb_min1) then
2365                            for i in 0 to 30 loop
2366                                k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_6_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).max;
2367    -- End of design block
2368                            end loop;
2369                        else
2370                            for i in 0 to 30 loop
2371                                k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_6_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).max;
2372    -- End of design block
2373                            end loop;
2374    -- End of design block
2375                        end if;
2376                    elsif(rgb_sync2.blue >= 170) then
2377                        if (rgb_sync2.red = rgb_min1) then
2378                            for i in 0 to 30 loop
2379                                k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
2380    -- End of design block
2381                            end loop;
2382                        else
2383                            for i in 0 to 30 loop
2384                                k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
2385    -- End of design block
2386                            end loop;
2387    -- End of design block
2388                        end if;
2389
                            ------------------------------------------------------------------------------
                            --------------------
2390                    elsif(rgb_sync2.red >= 85) then
2391
                            ------------------------------------------------------------------------------
                            --------------------
2392                        if (rgb_sync2.red = rgb_min1) then
2393                            for i in 0 to 30 loop
2394                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
2395    -- End of design block
2396                            end loop;
2397                        else
2398                            for i in 0 to 30 loop
2399                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
2400    -- End of design block
2401                            end loop;
2402    -- End of design block
2403                        end if;
2404
                            ------------------------------------------------------------------------------
                            --------------------
2405                    else
2406
                            ------------------------------------------------------------------------------
                            --------------------
2407                        if (rgb_sync2.red = rgb_min1) then
2408                              for i in 0 to 30 loop
2409                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
2410    -- End of design block
2411                              end loop;
2412                        else
2413                              for i in 0 to 30 loop
2414                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
2415    -- End of design block
2416                              end loop;
2417    -- End of design block
2418                        end if;
2419    -- End of design block
```

```
                end if;
-- End of design block
            end if;

            ------------------------------------------------------------------------------------------------------------------------------------
        elsif(centroid_lut_select = 6) then

            ------------------------------------------------------------------------------------------------------------------------------------
            if (rgb_sync2.red = rgb_max1) then
                if (rgb_sync2.red >= 170) then

                    ------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=   k_rgb_lut_3_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
-- End of design block
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=   k_rgb_lut_3_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
-- End of design block
                        end loop;
-- End of design block
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------------------
                elsif(rgb_sync2.red >= 85) then

                    ------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=   k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
-- End of design block
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=   k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
-- End of design block
                        end loop;
-- End of design block
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------------------
                else

                    ------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=   k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=   k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                        end loop;
-- End of design block
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------------------
```

```
-- End of design block
                end if;
            -- GREEN MAX
            elsif (rgb_sync2.green = rgb_max1) then
                if(rgb_sync2.green >= 170) then
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
-- End of design block
                          end loop;
-- End of design block
                    end if;

                    ------------------
```

```
                        end loop;
                    else
                        if ((abs(rgb_max - rgb_mid) <= 20) or (abs(rgb_max - rgb_min) <= 20))then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).min;
-- End of design block
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
-- End of design block
                            end loop;
-- End of design block
                        end if;
-- End of design block
                    end if;

                    ------------------
```

```
                                end loop;
-- End of design block
                            end if;
-- End of design block
                        end if;
-- End of design block
                    end if;
-- End of design block
                end if;

                ------------------
```

```
2640   -- End of design block
2641                           end loop;
2642   -- End of design block
2643                       end if;
2644
                           ------------------------------------------------------------------------------------------------------------------------
2645   -- End of design block
2646                   end if;
2647
                 ------------------------------------------------------------------------------------------------------------------------
2648               -- GREEN MAX
2649               elsif (rgb_sync2.green = rgb_max1) then
2650                   if(rgb_sync2.green >= 170) then
2651                       if (rgb_sync2.red = rgb_min1) then
2652                             for i in 0 to 30 loop
2653                               k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                   k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
2654   -- End of design block
2655                             end loop;
2656                       else
2657                             for i in 0 to 30 loop
2658                               k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                   k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
2659   -- End of design block
2660                             end loop;
2661   -- End of design block
2662                       end if;
2663
                           ------------------------------------------------------------------------------------------------------------------------
2664                   elsif(rgb_sync2.red >= 85) then
2665
                           ------------------------------------------------------------------------------------------------------------------------
2666                       if (rgb_sync2.red = rgb_min1) then
2667                             for i in 0 to 30 loop
2668                               k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                   k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2669   -- End of design block
2670                             end loop;
2671                       else
2672                             for i in 0 to 30 loop
2673                               k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                   k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2674   -- End of design block
2675                             end loop;
2676   -- End of design block
2677                       end if;
2678
                           ------------------------------------------------------------------------------------------------------------------------
2679                   else
2680
                           ------------------------------------------------------------------------------------------------------------------------
2681                       if (rgb_sync2.red = rgb_min1) then
2682                             for i in 0 to 30 loop
2683                               k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                   k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2684   -- End of design block
2685                             end loop;
2686                       else
2687                             for i in 0 to 30 loop
2688                               k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                   k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2689   -- End of design block
2690                             end loop;
2691   -- End of design block
2692                       end if;
2693   -- End of design block
2694                   end if;
2695               else
```

```
                ------------------------------------------------------------------------------------------------------------------
                -- BLUE MAX
                    if(rgb_sync2.blue >= 170) then
                        if (rgb_sync2.red = rgb_min1) then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
-- End of design block
                            end loop;
                        else
                            if ((abs(rgb_max - rgb_mid) <= 20) or (abs(rgb_max - rgb_min) <= 20))then
                                for i in 0 to 30 loop
                                    k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=
                                    k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).min;
-- End of design block
                                end loop;
                            else
                                for i in 0 to 30 loop
                                    k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
-- End of design block
                                end loop;
-- End of design block
                            end if;
-- End of design block
                        end if;

                        ------------------------------------------------------------------------------------------------------------------
                    elsif(rgb_sync2.red >= 85) then

                        ------------------------------------------------------------------------------------------------------------------
                        if (rgb_sync2.red = rgb_min1) then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                            end loop;
                        else
                            if ((abs(rgb_max - rgb_mid) <= 40) or (abs(rgb_max - rgb_min) <= 40))then
                                for i in 0 to 30 loop
                                    k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
-- End of design block
                                end loop;
                            else
                                for i in 0 to 30 loop
                                    k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
-- End of design block
                                end loop;
-- End of design block
                            end if;
-- End of design block
                        end if;

                        ------------------------------------------------------------------------------------------------------------------
                    else

                        ------------------------------------------------------------------------------------------------------------------
                        if (rgb_sync2.red = rgb_min1) then
                              for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                              end loop;
                        else
                            if ((abs(rgb_max - rgb_mid) <= 8) or (abs(rgb_max - rgb_min) <= 8))then
                                for i in 0 to 30 loop
```

```
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=  abs(k_rgb_lut_2_l(i).mid - 30);   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
-- End of design block
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
-- End of design block
                            end loop;
-- End of design block
                        end if;
-- End of design block
                    end if;
-- End of design block
                end if;
-- End of design block
            end if;

            ------------------
```

```
2804    -- End of design block
2805                            end loop;
2806    -- End of design block
2807                        end if;
2808
                            ------------------------------------------------------------------------------------------------------------------
2809                    else
2810
                            ------------------------------------------------------------------------------------------------------------------
2811                        if (rgb_sync2.green = rgb_min1) then
2812                            for i in 0 to 30 loop
2813                                k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2814    -- End of design block
2815                            end loop;
2816                        else
2817                            for i in 0 to 30 loop
2818                                k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                    k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2819    -- End of design block
2820                            end loop;
2821    -- End of design block
2822                        end if;
2823
                            ------------------------------------------------------------------------------------------------------------------
2824    -- End of design block
2825                    end if;
2826                -- GREEN MAX
2827                elsif (rgb_sync2.green = rgb_max1) then
2828                    if(rgb_sync2.green >= 170) then
2829                        if (rgb_sync2.red = rgb_min1) then
2830                              for i in 0 to 30 loop
2831                                k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
2832    -- End of design block
2833                              end loop;
2834                        else
2835                              for i in 0 to 30 loop
2836                                k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
2837    -- End of design block
2838                              end loop;
2839    -- End of design block
2840                        end if;
2841
                            ------------------------------------------------------------------------------------------------------------------
2842                    elsif(rgb_sync2.red >= 85) then
2843
                            ------------------------------------------------------------------------------------------------------------------
2844                        if (rgb_sync2.red = rgb_min1) then
2845                              for i in 0 to 30 loop
2846                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2847    -- End of design block
2848                              end loop;
2849                        else
2850                              for i in 0 to 30 loop
2851                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                    k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
2852    -- End of design block
2853                              end loop;
2854    -- End of design block
2855                        end if;
2856
                            ------------------------------------------------------------------------------------------------------------------
2857                    else
2858
                            ------------------------------------------------------------------------------------------------------------------
```

```
                    -------------------
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
-- End of design block
                          end loop;
-- End of design block
                    end if;
-- End of design block
                end if;
            else
            -- BLUE MAX
                if(rgb_sync2.blue >= 170) then
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
-- End of design block
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
-- End of design block
                        end loop;
-- End of design block
                    end if;

                    ------------------
```

```
2916   -- End of design block
2917                       end if;
2918   -- End of design block
2919                  end if;
2920   -- End of design block
2921              end if;
2922
                  ------------------------------------------------------------------------------------
                  -------------------
2923   -- End of design block
2924           end if;
2925   -- End of design block
2926       end if;
2927   -- End of design block
2928   end process;
2929   -- Sequential process block: executed on clock or sensitivity list events
2930   process (clk)begin
2931   -- Rising edge clock detection: ensures synchronous sequential logic
2932       if rising_edge(clk) then
2933         k1_rgb      <= k_rgb;
2934         k2_rgb      <= k1_rgb;
2935         k3_rgb      <= k2_rgb;
2936         k4_rgb      <= k3_rgb;
2937         k5_rgb      <= k4_rgb;
2938         k6_rgb      <= k5_rgb;
2939         k7_rgb      <= k6_rgb;
2940         k8_rgb      <= k7_rgb;
2941         k9_rgb      <= k8_rgb;
2942         k10rgb      <= k9_rgb;
2943         k11rgb      <= k10rgb;
2944         k12rgb      <= k11rgb;
2945         k13rgb      <= k12rgb;
2946         k14rgb      <= k13rgb;
2947         k15rgb      <= k14rgb;
2948         k16rgb      <= k15rgb;
2949         k17rgb      <= k16rgb;
2950         k18rgb      <= k17rgb;
2951         k19rgb      <= k18rgb;
2952         k20rgb      <= k19rgb;
2953         k21rgb      <= k20rgb;
2954         k22rgb      <= k21rgb;
2955         k23rgb      <= k22rgb;
2956         k24rgb      <= k23rgb;
2957         k25rgb      <= k24rgb;
2958         k26rgb      <= k25rgb;
2959         k27rgb      <= k26rgb;
2960   -- End of design block
2961       end if;
2962   -- End of design block
2963   end process;
2964   -- Sequential process block: executed on clock or sensitivity list events
2965   process (clk)begin
2966   -- Rising edge clock detection: ensures synchronous sequential logic
2967       if rising_edge(clk) then
2968           rgb(1).red     <= std_logic_vector(to_unsigned(k26rgb(1).red, 8)) & "00";
2969           rgb(1).gre     <= std_logic_vector(to_unsigned(k26rgb(1).gre, 8)) & "00";
2970           rgb(1).blu     <= std_logic_vector(to_unsigned(k26rgb(1).blu, 8)) & "00";
2971           rgb(2).red     <= std_logic_vector(to_unsigned(k26rgb(2).red, 8)) & "00";
2972           rgb(2).gre     <= std_logic_vector(to_unsigned(k26rgb(2).gre, 8)) & "00";
2973           rgb(2).blu     <= std_logic_vector(to_unsigned(k26rgb(2).blu, 8)) & "00";
2974           rgb(3).red     <= std_logic_vector(to_unsigned(k26rgb(3).red, 8)) & "00";
2975           rgb(3).gre     <= std_logic_vector(to_unsigned(k26rgb(3).gre, 8)) & "00";
2976           rgb(3).blu     <= std_logic_vector(to_unsigned(k26rgb(3).blu, 8)) & "00";
2977           rgb(4).red     <= std_logic_vector(to_unsigned(k26rgb(4).red, 8)) & "00";
2978           rgb(4).gre     <= std_logic_vector(to_unsigned(k26rgb(4).gre, 8)) & "00";
2979           rgb(4).blu     <= std_logic_vector(to_unsigned(k26rgb(4).blu, 8)) & "00";
2980           rgb(5).red     <= std_logic_vector(to_unsigned(k26rgb(5).red, 8)) & "00";
2981           rgb(5).gre     <= std_logic_vector(to_unsigned(k26rgb(5).gre, 8)) & "00";
2982           rgb(5).blu     <= std_logic_vector(to_unsigned(k26rgb(5).blu, 8)) & "00";
2983           rgb(6).red     <= std_logic_vector(to_unsigned(k26rgb(6).red, 8)) & "00";
2984           rgb(6).gre     <= std_logic_vector(to_unsigned(k26rgb(6).gre, 8)) & "00";
2985           rgb(6).blu     <= std_logic_vector(to_unsigned(k26rgb(6).blu, 8)) & "00";
2986           rgb(7).red     <= std_logic_vector(to_unsigned(k26rgb(7).red, 8)) & "00";
2987           rgb(7).gre     <= std_logic_vector(to_unsigned(k26rgb(7).gre, 8)) & "00";
```

```
2988            rgb(7).blu     <= std_logic_vector(to_unsigned(k26rgb(7).blu, 8)) & "00";
2989            rgb(8).red     <= std_logic_vector(to_unsigned(k26rgb(8).red, 8)) & "00";
2990            rgb(8).gre     <= std_logic_vector(to_unsigned(k26rgb(8).gre, 8)) & "00";
2991            rgb(8).blu     <= std_logic_vector(to_unsigned(k26rgb(8).blu, 8)) & "00";
2992            rgb(9).red     <= std_logic_vector(to_unsigned(k26rgb(9).red, 8)) & "00";
2993            rgb(9).gre     <= std_logic_vector(to_unsigned(k26rgb(9).gre, 8)) & "00";
2994            rgb(9).blu     <= std_logic_vector(to_unsigned(k26rgb(9).blu, 8)) & "00";
2995            rgb(10).red    <= std_logic_vector(to_unsigned(k26rgb(10).red, 8)) & "00";
2996            rgb(10).gre    <= std_logic_vector(to_unsigned(k26rgb(10).gre, 8)) & "00";
2997            rgb(10).blu    <= std_logic_vector(to_unsigned(k26rgb(10).blu, 8)) & "00";
2998            rgb(11).red    <= std_logic_vector(to_unsigned(k26rgb(11).red, 8)) & "00";
2999            rgb(11).gre    <= std_logic_vector(to_unsigned(k26rgb(11).gre, 8)) & "00";
3000            rgb(11).blu    <= std_logic_vector(to_unsigned(k26rgb(11).blu, 8)) & "00";
3001            rgb(12).red    <= std_logic_vector(to_unsigned(k26rgb(12).red, 8)) & "00";
3002            rgb(12).gre    <= std_logic_vector(to_unsigned(k26rgb(12).gre, 8)) & "00";
3003            rgb(12).blu    <= std_logic_vector(to_unsigned(k26rgb(12).blu, 8)) & "00";
3004            rgb(13).red    <= std_logic_vector(to_unsigned(k26rgb(13).red, 8)) & "00";
3005            rgb(13).gre    <= std_logic_vector(to_unsigned(k26rgb(13).gre, 8)) & "00";
3006            rgb(13).blu    <= std_logic_vector(to_unsigned(k26rgb(13).blu, 8)) & "00";
3007            rgb(14).red    <= std_logic_vector(to_unsigned(k26rgb(14).red, 8)) & "00";
3008            rgb(14).gre    <= std_logic_vector(to_unsigned(k26rgb(14).gre, 8)) & "00";
3009            rgb(14).blu    <= std_logic_vector(to_unsigned(k26rgb(14).blu, 8)) & "00";
3010            rgb(15).red    <= std_logic_vector(to_unsigned(k26rgb(15).red, 8)) & "00";
3011            rgb(15).gre    <= std_logic_vector(to_unsigned(k26rgb(15).gre, 8)) & "00";
3012            rgb(15).blu    <= std_logic_vector(to_unsigned(k26rgb(15).blu, 8)) & "00";
3013            rgb(16).red    <= std_logic_vector(to_unsigned(k26rgb(16).red, 8)) & "00";
3014            rgb(16).gre    <= std_logic_vector(to_unsigned(k26rgb(16).gre, 8)) & "00";
3015            rgb(16).blu    <= std_logic_vector(to_unsigned(k26rgb(16).blu, 8)) & "00";
3016            rgb(17).red    <= std_logic_vector(to_unsigned(k26rgb(17).red, 8)) & "00";
3017            rgb(17).gre    <= std_logic_vector(to_unsigned(k26rgb(17).gre, 8)) & "00";
3018            rgb(17).blu    <= std_logic_vector(to_unsigned(k26rgb(17).blu, 8)) & "00";
3019            rgb(18).red    <= std_logic_vector(to_unsigned(k26rgb(18).red, 8)) & "00";
3020            rgb(18).gre    <= std_logic_vector(to_unsigned(k26rgb(18).gre, 8)) & "00";
3021            rgb(18).blu    <= std_logic_vector(to_unsigned(k26rgb(18).blu, 8)) & "00";
3022            rgb(19).red    <= std_logic_vector(to_unsigned(k26rgb(19).red, 8)) & "00";
3023            rgb(19).gre    <= std_logic_vector(to_unsigned(k26rgb(19).gre, 8)) & "00";
3024            rgb(19).blu    <= std_logic_vector(to_unsigned(k26rgb(19).blu, 8)) & "00";
3025            rgb(20).red    <= std_logic_vector(to_unsigned(k26rgb(20).red, 8)) & "00";
3026            rgb(20).gre    <= std_logic_vector(to_unsigned(k26rgb(20).gre, 8)) & "00";
3027            rgb(20).blu    <= std_logic_vector(to_unsigned(k26rgb(20).blu, 8)) & "00";
3028            rgb(21).red    <= std_logic_vector(to_unsigned(k26rgb(21).red, 8)) & "00";
3029            rgb(21).gre    <= std_logic_vector(to_unsigned(k26rgb(21).gre, 8)) & "00";
3030            rgb(21).blu    <= std_logic_vector(to_unsigned(k26rgb(21).blu, 8)) & "00";
3031            rgb(22).red    <= std_logic_vector(to_unsigned(k26rgb(22).red, 8)) & "00";
3032            rgb(22).gre    <= std_logic_vector(to_unsigned(k26rgb(22).gre, 8)) & "00";
3033            rgb(22).blu    <= std_logic_vector(to_unsigned(k26rgb(22).blu, 8)) & "00";
3034            rgb(23).red    <= std_logic_vector(to_unsigned(k26rgb(23).red, 8)) & "00";
3035            rgb(23).gre    <= std_logic_vector(to_unsigned(k26rgb(23).gre, 8)) & "00";
3036            rgb(23).blu    <= std_logic_vector(to_unsigned(k26rgb(23).blu, 8)) & "00";
3037            rgb(24).red    <= std_logic_vector(to_unsigned(k26rgb(24).red, 8)) & "00";
3038            rgb(24).gre    <= std_logic_vector(to_unsigned(k26rgb(24).gre, 8)) & "00";
3039            rgb(24).blu    <= std_logic_vector(to_unsigned(k26rgb(24).blu, 8)) & "00";
3040            rgb(25).red    <= std_logic_vector(to_unsigned(k26rgb(25).red, 8)) & "00";
3041            rgb(25).gre    <= std_logic_vector(to_unsigned(k26rgb(25).gre, 8)) & "00";
3042            rgb(25).blu    <= std_logic_vector(to_unsigned(k26rgb(25).blu, 8)) & "00";
3043            rgb(26).red    <= std_logic_vector(to_unsigned(k26rgb(26).red, 8)) & "00";
3044            rgb(26).gre    <= std_logic_vector(to_unsigned(k26rgb(26).gre, 8)) & "00";
3045            rgb(26).blu    <= std_logic_vector(to_unsigned(k26rgb(26).blu, 8)) & "00";
3046            rgb(27).red    <= std_logic_vector(to_unsigned(k26rgb(27).red, 8)) & "00";
3047            rgb(27).gre    <= std_logic_vector(to_unsigned(k26rgb(27).gre, 8)) & "00";
3048            rgb(27).blu    <= std_logic_vector(to_unsigned(k26rgb(27).blu, 8)) & "00";
3049            rgb(28).red    <= std_logic_vector(to_unsigned(k26rgb(28).red, 8)) & "00";
3050            rgb(28).gre    <= std_logic_vector(to_unsigned(k26rgb(28).gre, 8)) & "00";
3051            rgb(28).blu    <= std_logic_vector(to_unsigned(k26rgb(28).blu, 8)) & "00";
3052            rgb(29).red    <= std_logic_vector(to_unsigned(k26rgb(29).red, 8)) & "00";
3053            rgb(29).gre    <= std_logic_vector(to_unsigned(k26rgb(29).gre, 8)) & "00";
3054            rgb(29).blu    <= std_logic_vector(to_unsigned(k26rgb(29).blu, 8)) & "00";
3055            rgb(30).red    <= std_logic_vector(to_unsigned(k26rgb(30).red, 8)) & "00";
3056            rgb(30).gre    <= std_logic_vector(to_unsigned(k26rgb(30).gre, 8)) & "00";
3057            rgb(30).blu    <= std_logic_vector(to_unsigned(k26rgb(30).blu, 8)) & "00";
3058    -- End of design block
3059        end if;
3060    -- End of design block
3061    end process;
```

```
3062  color_k1_clustering_inst: rgb_cluster_core
3063  -- Generic parameters: compile□time configuration values
3064  generic map(
3065      data_width        => i_data_width)
3066  -- Port definitions: input/output interface signals
3067  port map(
3068      clk               => clk,
3069      rst_n             => rst_n,
3070      pixel_in_rgb              => rgb_std4sync,
3071      k_rgb.red         => k_rgb(1).red,
3072      k_rgb.gre         => k_rgb(1).gre,
3073      k_rgb.blu         => k_rgb(1).blu,
3074      threshold         => thr1.threshold1);
3075  color_k2_clustering_inst: rgb_cluster_core
3076  -- Generic parameters: compile□time configuration values
3077  generic map(
3078      data_width        => i_data_width)
3079  -- Port definitions: input/output interface signals
3080  port map(
3081      clk               => clk,
3082      rst_n             => rst_n,
3083      pixel_in_rgb              => rgb_std4sync,
3084      k_rgb.red         => k_rgb(2).red,
3085      k_rgb.gre         => k_rgb(2).gre,
3086      k_rgb.blu         => k_rgb(2).blu,
3087      threshold         => thr1.threshold2);
3088  color_k3_clustering_inst: rgb_cluster_core
3089  -- Generic parameters: compile□time configuration values
3090  generic map(
3091      data_width        => i_data_width)
3092  -- Port definitions: input/output interface signals
3093  port map(
3094      clk               => clk,
3095      rst_n             => rst_n,
3096      pixel_in_rgb              => rgb_std4sync,
3097      k_rgb.red         => k_rgb(3).red,
3098      k_rgb.gre         => k_rgb(3).gre,
3099      k_rgb.blu         => k_rgb(3).blu,
3100      threshold         => thr1.threshold3);
3101  color_k4_clustering_inst: rgb_cluster_core
3102  -- Generic parameters: compile□time configuration values
3103  generic map(
3104      data_width        => i_data_width)
3105  -- Port definitions: input/output interface signals
3106  port map(
3107      clk               => clk,
3108      rst_n             => rst_n,
3109      pixel_in_rgb              => rgb_std4sync,
3110      k_rgb.red         => k_rgb(4).red,
3111      k_rgb.gre         => k_rgb(4).gre,
3112      k_rgb.blu         => k_rgb(4).blu,
3113      threshold         => thr1.threshold4);
3114  color_k5_clustering_inst: rgb_cluster_core
3115  -- Generic parameters: compile□time configuration values
3116  generic map(
3117      data_width        => i_data_width)
3118  -- Port definitions: input/output interface signals
3119  port map(
3120      clk               => clk,
3121      rst_n             => rst_n,
3122      pixel_in_rgb              => rgb_std4sync,
3123      k_rgb.red         => k_rgb(5).red,
3124      k_rgb.gre         => k_rgb(5).gre,
3125      k_rgb.blu         => k_rgb(5).blu,
3126      threshold         => thr1.threshold5);
3127  color_k6_clustering_inst: rgb_cluster_core
3128  -- Generic parameters: compile□time configuration values
3129  generic map(
3130      data_width        => i_data_width)
3131  -- Port definitions: input/output interface signals
3132  port map(
3133      clk               => clk,
3134      rst_n             => rst_n,
3135      pixel_in_rgb              => rgb_std4sync,
```

```
3136        k_rgb.red          => k_rgb(6).red,
3137        k_rgb.gre          => k_rgb(6).gre,
3138        k_rgb.blu          => k_rgb(6).blu,
3139        threshold          => thr1.threshold6);
3140    color_k7_clustering_inst: rgb_cluster_core
3141    -- Generic parameters: compile□time configuration values
3142    generic map(
3143        data_width        => i_data_width)
3144    -- Port definitions: input/output interface signals
3145    port map(
3146        clk                => clk,
3147        rst_n              => rst_n,
3148        pixel_in_rgb               => rgb_std4sync,
3149        k_rgb.red          => k_rgb(7).red,
3150        k_rgb.gre          => k_rgb(7).gre,
3151        k_rgb.blu          => k_rgb(7).blu,
3152        threshold          => thr1.threshold7);
3153    color_k8_clustering_inst: rgb_cluster_core
3154    -- Generic parameters: compile□time configuration values
3155    generic map(
3156        data_width        => i_data_width)
3157    -- Port definitions: input/output interface signals
3158    port map(
3159        clk                => clk,
3160        rst_n              => rst_n,
3161        pixel_in_rgb               => rgb_std4sync,
3162        k_rgb.red          => k_rgb(8).red,
3163        k_rgb.gre          => k_rgb(8).gre,
3164        k_rgb.blu          => k_rgb(8).blu,
3165        threshold          => thr1.threshold8);
3166    color_k9_clustering_inst: rgb_cluster_core
3167    -- Generic parameters: compile□time configuration values
3168    generic map(
3169        data_width        => i_data_width)
3170    -- Port definitions: input/output interface signals
3171    port map(
3172        clk                => clk,
3173        rst_n              => rst_n,
3174        pixel_in_rgb               => rgb_std4sync,
3175        k_rgb.red          => k_rgb(9).red,
3176        k_rgb.gre          => k_rgb(9).gre,
3177        k_rgb.blu          => k_rgb(9).blu,
3178        threshold          => thr1.threshold9);
3179    color_k10_clustering_inst: rgb_cluster_core
3180    -- Generic parameters: compile□time configuration values
3181    generic map(
3182        data_width        => i_data_width)
3183    -- Port definitions: input/output interface signals
3184    port map(
3185        clk                => clk,
3186        rst_n              => rst_n,
3187        pixel_in_rgb               => rgb_std4sync,
3188        k_rgb.red          => k_rgb(10).red,
3189        k_rgb.gre          => k_rgb(10).gre,
3190        k_rgb.blu          => k_rgb(10).blu,
3191        threshold          => thr1.threshold10);
3192    color_k11_clustering_inst: rgb_cluster_core
3193    -- Generic parameters: compile□time configuration values
3194    generic map(
3195        data_width        => i_data_width)
3196    -- Port definitions: input/output interface signals
3197    port map(
3198        clk                => clk,
3199        rst_n              => rst_n,
3200        pixel_in_rgb               => rgb_std4sync,
3201        k_rgb.red          => k_rgb(11).red,
3202        k_rgb.gre          => k_rgb(11).gre,
3203        k_rgb.blu          => k_rgb(11).blu,
3204        threshold          => thr1.threshold11);
3205    color_k12_clustering_inst: rgb_cluster_core
3206    -- Generic parameters: compile□time configuration values
3207    generic map(
3208        data_width        => i_data_width)
3209    -- Port definitions: input/output interface signals
```

```
3210  port map(
3211      clk                 => clk,
3212      rst_n               => rst_n,
3213      pixel_in_rgb                => rgb_std4sync,
3214      k_rgb.red           => k_rgb(12).red,
3215      k_rgb.gre           => k_rgb(12).gre,
3216      k_rgb.blu           => k_rgb(12).blu,
3217      threshold           => thr1.threshold12);
3218  color_k13_clustering_inst: rgb_cluster_core
3219  -- Generic parameters: compile time configuration values
3220  generic map(
3221      data_width        => i_data_width)
3222  -- Port definitions: input/output interface signals
3223  port map(
3224      clk                 => clk,
3225      rst_n               => rst_n,
3226      pixel_in_rgb                => rgb_std4sync,
3227      k_rgb.red           => k_rgb(13).red,
3228      k_rgb.gre           => k_rgb(13).gre,
3229      k_rgb.blu           => k_rgb(13).blu,
3230      threshold           => thr1.threshold13);
3231  color_k14_clustering_inst: rgb_cluster_core
3232  -- Generic parameters: compile time configuration values
3233  generic map(
3234      data_width        => i_data_width)
3235  -- Port definitions: input/output interface signals
3236  port map(
3237      clk                 => clk,
3238      rst_n               => rst_n,
3239      pixel_in_rgb                => rgb_std4sync,
3240      k_rgb.red           => k_rgb(14).red,
3241      k_rgb.gre           => k_rgb(14).gre,
3242      k_rgb.blu           => k_rgb(14).blu,
3243      threshold           => thr1.threshold14);
3244  color_k15_clustering_inst: rgb_cluster_core
3245  -- Generic parameters: compile time configuration values
3246  generic map(
3247      data_width        => i_data_width)
3248  -- Port definitions: input/output interface signals
3249  port map(
3250      clk                 => clk,
3251      rst_n               => rst_n,
3252      pixel_in_rgb                => rgb_std4sync,
3253      k_rgb.red           => k_rgb(15).red,
3254      k_rgb.gre           => k_rgb(15).gre,
3255      k_rgb.blu           => k_rgb(15).blu,
3256      threshold           => thr1.threshold15);
3257  color_k16clustering_inst: rgb_cluster_core
3258  -- Generic parameters: compile time configuration values
3259  generic map(
3260      data_width        => i_data_width)
3261  -- Port definitions: input/output interface signals
3262  port map(
3263      clk                 => clk,
3264      rst_n               => rst_n,
3265      pixel_in_rgb                => rgb_std4sync,
3266      k_rgb.red           => k_rgb(16).red,
3267      k_rgb.gre           => k_rgb(16).gre,
3268      k_rgb.blu           => k_rgb(16).blu,
3269      threshold           => thr1.threshold16);
3270  color_k17_clustering_inst: rgb_cluster_core
3271  -- Generic parameters: compile time configuration values
3272  generic map(
3273      data_width        => i_data_width)
3274  -- Port definitions: input/output interface signals
3275  port map(
3276      clk                 => clk,
3277      rst_n               => rst_n,
3278      pixel_in_rgb                => rgb_std4sync,
3279      k_rgb.red           => k_rgb(17).red,
3280      k_rgb.gre           => k_rgb(17).gre,
3281      k_rgb.blu           => k_rgb(17).blu,
3282      threshold           => thr1.threshold17);
3283  color_k18_clustering_inst: rgb_cluster_core
```

```
3284  -- Generic parameters: compile□time configuration values
3285  generic map(
3286      data_width        => i_data_width)
3287  -- Port definitions: input/output interface signals
3288  port map(
3289      clk               => clk,
3290      rst_n             => rst_n,
3291      pixel_in_rgb              => rgb_std4sync,
3292      k_rgb.red         => k_rgb(18).red,
3293      k_rgb.gre         => k_rgb(18).gre,
3294      k_rgb.blu         => k_rgb(18).blu,
3295      threshold         => thr1.threshold18);
3296  color_k19_clustering_inst: rgb_cluster_core
3297  -- Generic parameters: compile□time configuration values
3298  generic map(
3299      data_width        => i_data_width)
3300  -- Port definitions: input/output interface signals
3301  port map(
3302      clk               => clk,
3303      rst_n             => rst_n,
3304      pixel_in_rgb              => rgb_std4sync,
3305      k_rgb.red         => k_rgb(19).red,
3306      k_rgb.gre         => k_rgb(19).gre,
3307      k_rgb.blu         => k_rgb(19).blu,
3308      threshold         => thr1.threshold19);
3309  color_k20_clustering_inst: rgb_cluster_core
3310  -- Generic parameters: compile□time configuration values
3311  generic map(
3312      data_width        => i_data_width)
3313  -- Port definitions: input/output interface signals
3314  port map(
3315      clk               => clk,
3316      rst_n             => rst_n,
3317      pixel_in_rgb              => rgb_std4sync,
3318      k_rgb.red         => k_rgb(20).red,
3319      k_rgb.gre         => k_rgb(20).gre,
3320      k_rgb.blu         => k_rgb(20).blu,
3321      threshold         => thr1.threshold20);
3322  color_k21_clustering_inst: rgb_cluster_core
3323  -- Generic parameters: compile□time configuration values
3324  generic map(
3325      data_width        => i_data_width)
3326  -- Port definitions: input/output interface signals
3327  port map(
3328      clk               => clk,
3329      rst_n             => rst_n,
3330      pixel_in_rgb              => rgb_std4sync,
3331      k_rgb.red         => k_rgb(21).red,
3332      k_rgb.gre         => k_rgb(21).gre,
3333      k_rgb.blu         => k_rgb(21).blu,
3334      threshold         => thr1.threshold21);
3335  color_k22_clustering_inst: rgb_cluster_core
3336  -- Generic parameters: compile□time configuration values
3337  generic map(
3338      data_width        => i_data_width)
3339  -- Port definitions: input/output interface signals
3340  port map(
3341      clk               => clk,
3342      rst_n             => rst_n,
3343      pixel_in_rgb              => rgb_std4sync,
3344      k_rgb.red         => k_rgb(22).red,
3345      k_rgb.gre         => k_rgb(22).gre,
3346      k_rgb.blu         => k_rgb(22).blu,
3347      threshold         => thr1.threshold22);
3348  color_k23_clustering_inst: rgb_cluster_core
3349  -- Generic parameters: compile□time configuration values
3350  generic map(
3351      data_width        => i_data_width)
3352  -- Port definitions: input/output interface signals
3353  port map(
3354      clk               => clk,
3355      rst_n             => rst_n,
3356      pixel_in_rgb              => rgb_std4sync,
3357      k_rgb.red         => k_rgb(23).red,
```

```
3358        k_rgb.gre          => k_rgb(23).gre,
3359        k_rgb.blu          => k_rgb(23).blu,
3360        threshold          => thr1.threshold23);
3361    color_k24_clustering_inst: rgb_cluster_core
3362    -- Generic parameters: compile□time configuration values
3363    generic map(
3364        data_width        => i_data_width)
3365    -- Port definitions: input/output interface signals
3366    port map(
3367        clk                => clk,
3368        rst_n              => rst_n,
3369        pixel_in_rgb               => rgb_std4sync,
3370        k_rgb.red          => k_rgb(24).red,
3371        k_rgb.gre          => k_rgb(24).gre,
3372        k_rgb.blu          => k_rgb(24).blu,
3373        threshold          => thr1.threshold24);
3374    color_k25_clustering_inst: rgb_cluster_core
3375    -- Generic parameters: compile□time configuration values
3376    generic map(
3377        data_width        => i_data_width)
3378    -- Port definitions: input/output interface signals
3379    port map(
3380        clk                => clk,
3381        rst_n              => rst_n,
3382        pixel_in_rgb               => rgb_std4sync,
3383        k_rgb.red          => k_rgb(25).red,
3384        k_rgb.gre          => k_rgb(25).gre,
3385        k_rgb.blu          => k_rgb(25).blu,
3386        threshold          => thr1.threshold25);
3387    color_k26_clustering_inst: rgb_cluster_core
3388    -- Generic parameters: compile□time configuration values
3389    generic map(
3390        data_width        => i_data_width)
3391    -- Port definitions: input/output interface signals
3392    port map(
3393        clk                => clk,
3394        rst_n              => rst_n,
3395        pixel_in_rgb               => rgb_std4sync,
3396        k_rgb.red          => k_rgb(26).red,
3397        k_rgb.gre          => k_rgb(26).gre,
3398        k_rgb.blu          => k_rgb(26).blu,
3399        threshold          => thr1.threshold26);
3400    color_k27_clustering_inst: rgb_cluster_core
3401    -- Generic parameters: compile□time configuration values
3402    generic map(
3403        data_width        => i_data_width)
3404    -- Port definitions: input/output interface signals
3405    port map(
3406        clk                => clk,
3407        rst_n              => rst_n,
3408        pixel_in_rgb               => rgb_std4sync,
3409        k_rgb.red          => k_rgb(27).red,
3410        k_rgb.gre          => k_rgb(27).gre,
3411        k_rgb.blu          => k_rgb(27).blu,
3412        threshold          => thr1.threshold27);
3413    color_k28_clustering_inst: rgb_cluster_core
3414    -- Generic parameters: compile□time configuration values
3415    generic map(
3416        data_width        => i_data_width)
3417    -- Port definitions: input/output interface signals
3418    port map(
3419        clk                => clk,
3420        rst_n              => rst_n,
3421        pixel_in_rgb               => rgb_std4sync,
3422        k_rgb.red          => k_rgb(28).red,
3423        k_rgb.gre          => k_rgb(28).gre,
3424        k_rgb.blu          => k_rgb(28).blu,
3425        threshold          => thr1.threshold28);
3426    color_k29_clustering_inst: rgb_cluster_core
3427    -- Generic parameters: compile□time configuration values
3428    generic map(
3429        data_width        => i_data_width)
3430    -- Port definitions: input/output interface signals
3431    port map(
```

```
3432        clk                 => clk,
3433        rst_n               => rst_n,
3434        pixel_in_rgb         => rgb_std4sync,
3435        k_rgb.red           => k_rgb(29).red,
3436        k_rgb.gre           => k_rgb(29).gre,
3437        k_rgb.blu           => k_rgb(29).blu,
3438        threshold           => thr1.threshold29);
3439    color_k30_clustering_inst: rgb_cluster_core
3440    -- Generic parameters: compile□time configuration values
3441    generic map(
3442        data_width        => i_data_width)
3443    -- Port definitions: input/output interface signals
3444    port map(
3445        clk                 => clk,
3446        rst_n               => rst_n,
3447        pixel_in_rgb                => rgb_std4sync,
3448        k_rgb.red           => k_rgb(30).red,
3449        k_rgb.gre           => k_rgb(30).gre,
3450        k_rgb.blu           => k_rgb(30).blu,
3451        threshold           => thr1.threshold30);
3452    -- Sequential process block: executed on clock or sensitivity list events
3453    process (clk) begin
3454    -- Rising edge clock detection: ensures synchronous sequential logic
3455        if rising_edge(clk) then
3456            threshold_lms_1  <= int_min_val(thr1.threshold1,thr1.threshold2,thr1.threshold3,thr1.
                threshold4,thr1.threshold5);
3457            threshold_lms_2  <= int_min_val(thr1.threshold6,thr1.threshold7,thr1.threshold8,thr1.
                threshold9,thr1.threshold10);
3458            threshold_lms_3  <= int_min_val(thr1.threshold11,thr1.threshold12,thr1.threshold13,thr1.
                threshold14,thr1.threshold15);
3459            threshold_lms_4  <= int_min_val(thr1.threshold16,thr1.threshold17,thr1.threshold18,thr1.
                threshold19,thr1.threshold20);
3460            threshold_lms_5  <= int_min_val(thr1.threshold21,thr1.threshold22,thr1.threshold23,thr1.
                threshold24,thr1.threshold25);
3461            threshold_lms_6  <= int_min_val(thr1.threshold26,thr1.threshold27,thr1.threshold28,thr1.
                threshold29,thr1.threshold30);
3462    -- End of design block
3463        end if;
3464    -- End of design block
3465    end process;
3466    -- Sequential process block: executed on clock or sensitivity list events
3467    process (clk) begin
3468    -- Rising edge clock detection: ensures synchronous sequential logic
3469        if rising_edge(clk) then
3470            threshold_red1   <= int_min_val(threshold_lms_1,threshold_lms_2,threshold_lms_3);
3471            threshold_red2   <= int_min_val(threshold_lms_4,threshold_lms_5,threshold_lms_6);
3472    -- End of design block
3473        end if;
3474    -- End of design block
3475    end process;
3476    -- Sequential process block: executed on clock or sensitivity list events
3477    process (clk) begin
3478    -- Rising edge clock detection: ensures synchronous sequential logic
3479        if rising_edge(clk) then
3480            threshold_lms    <= int_min_val(threshold_red1,threshold_red2);
3481            threshold        <= threshold_lms;
3482    -- End of design block
3483        end if;
3484    -- End of design block
3485    end process;
3486    -- Sequential process block: executed on clock or sensitivity list events
3487    process (clk) begin
3488    -- Rising edge clock detection: ensures synchronous sequential logic
3489        if rising_edge(clk) then
3490            thr2 <= thr1;
3491            thr3 <= thr2;
3492            thr4 <= thr3;
3493            thr5 <= thr4;
3494    -- End of design block
3495        end if;
3496    -- End of design block
3497    end process;
3498    -- Sequential process block: executed on clock or sensitivity list events
3499    process (clk) begin
```

```
3500   -- Rising edge clock detection: ensures synchronous sequential logic
3501       if rising_edge(clk) then
3502           if ((thr5.threshold1  = threshold)) then
3503               pixel_out_rgb.red     <= rgb(1).red;
3504               pixel_out_rgb.green   <= rgb(1).gre;
3505               pixel_out_rgb.blue    <= rgb(1).blu;
3506           elsif((thr5.threshold2 = threshold)) then
3507               pixel_out_rgb.red     <= rgb(2).red;
3508               pixel_out_rgb.green   <= rgb(2).gre;
3509               pixel_out_rgb.blue    <= rgb(2).blu;
3510           elsif((thr5.threshold3 = threshold)) then
3511               pixel_out_rgb.red     <= rgb(3).red;
3512               pixel_out_rgb.green   <= rgb(3).gre;
3513               pixel_out_rgb.blue    <= rgb(3).blu;
3514           elsif((thr5.threshold4 = threshold)) then
3515               pixel_out_rgb.red     <= rgb(4).red;
3516               pixel_out_rgb.green   <= rgb(4).gre;
3517               pixel_out_rgb.blue    <= rgb(4).blu;
3518           elsif((thr5.threshold5 = threshold)) then
3519               pixel_out_rgb.red     <= rgb(5).red;
3520               pixel_out_rgb.green   <= rgb(5).gre;
3521               pixel_out_rgb.blue    <= rgb(5).blu;
3522           elsif((thr5.threshold6 = threshold)) then
3523               pixel_out_rgb.red     <= rgb(6).red;
3524               pixel_out_rgb.green   <= rgb(6).gre;
3525               pixel_out_rgb.blue    <= rgb(6).blu;
3526           elsif((thr5.threshold7 = threshold)) then
3527               pixel_out_rgb.red     <= rgb(7).red;
3528               pixel_out_rgb.green   <= rgb(7).gre;
3529               pixel_out_rgb.blue    <= rgb(7).blu;
3530           elsif((thr5.threshold8 = threshold)) then
3531               pixel_out_rgb.red     <= rgb(8).red;
3532               pixel_out_rgb.green   <= rgb(8).gre;
3533               pixel_out_rgb.blue    <= rgb(8).blu;
3534           elsif((thr5.threshold9 = threshold)) then
3535               pixel_out_rgb.red     <= rgb(9).red;
3536               pixel_out_rgb.green   <= rgb(9).gre;
3537               pixel_out_rgb.blue    <= rgb(9).blu;
3538           elsif((thr5.threshold10 = threshold)) then
3539               pixel_out_rgb.red     <= rgb(10).red;
3540               pixel_out_rgb.green   <= rgb(10).gre;
3541               pixel_out_rgb.blue    <= rgb(10).blu;
3542           elsif((thr5.threshold11 = threshold)) then
3543               pixel_out_rgb.red     <= rgb(11).red;
3544               pixel_out_rgb.green   <= rgb(11).gre;
3545               pixel_out_rgb.blue    <= rgb(11).blu;
3546           elsif((thr5.threshold12 = threshold)) then
3547               pixel_out_rgb.red     <= rgb(12).red;
3548               pixel_out_rgb.green   <= rgb(12).gre;
3549               pixel_out_rgb.blue    <= rgb(12).blu;
3550           elsif((thr5.threshold13 = threshold)) then
3551               pixel_out_rgb.red     <= rgb(13).red;
3552               pixel_out_rgb.green   <= rgb(13).gre;
3553               pixel_out_rgb.blue    <= rgb(13).blu;
3554           elsif((thr5.threshold14 = threshold)) then
3555               pixel_out_rgb.red     <= rgb(14).red;
3556               pixel_out_rgb.green   <= rgb(14).gre;
3557               pixel_out_rgb.blue    <= rgb(14).blu;
3558           elsif((thr5.threshold15 = threshold)) then
3559               pixel_out_rgb.red     <= rgb(15).red;
3560               pixel_out_rgb.green   <= rgb(15).gre;
3561               pixel_out_rgb.blue    <= rgb(15).blu;
3562           elsif((thr5.threshold16 = threshold)) then
3563               pixel_out_rgb.red     <= rgb(16).red;
3564               pixel_out_rgb.green   <= rgb(16).gre;
3565               pixel_out_rgb.blue    <= rgb(16).blu;
3566           elsif((thr5.threshold17 = threshold)) then
3567               pixel_out_rgb.red     <= rgb(17).red;
3568               pixel_out_rgb.green   <= rgb(17).gre;
3569               pixel_out_rgb.blue    <= rgb(17).blu;
3570           elsif((thr5.threshold18 = threshold)) then
3571               pixel_out_rgb.red     <= rgb(18).red;
3572               pixel_out_rgb.green   <= rgb(18).gre;
3573               pixel_out_rgb.blue    <= rgb(18).blu;
```

```
3574            elsif((thr5.threshold19 = threshold)) then
3575                pixel_out_rgb.red     <= rgb(19).red;
3576                pixel_out_rgb.green   <= rgb(19).gre;
3577                pixel_out_rgb.blue    <= rgb(19).blu;
3578            elsif((thr5.threshold20 = threshold)) then
3579                pixel_out_rgb.red     <= rgb(20).red;
3580                pixel_out_rgb.green   <= rgb(20).gre;
3581                pixel_out_rgb.blue    <= rgb(20).blu;
3582            elsif((thr5.threshold21 = threshold)) then
3583                pixel_out_rgb.red     <= rgb(21).red;
3584                pixel_out_rgb.green   <= rgb(21).gre;
3585                pixel_out_rgb.blue    <= rgb(21).blu;
3586            elsif((thr5.threshold22 = threshold)) then
3587                pixel_out_rgb.red     <= rgb(22).red;
3588                pixel_out_rgb.green   <= rgb(22).gre;
3589                pixel_out_rgb.blue    <= rgb(22).blu;
3590            elsif((thr5.threshold23 = threshold)) then
3591                pixel_out_rgb.red     <= rgb(23).red;
3592                pixel_out_rgb.green   <= rgb(23).gre;
3593                pixel_out_rgb.blue    <= rgb(23).blu;
3594            elsif((thr5.threshold24 = threshold)) then
3595                pixel_out_rgb.red     <= rgb(24).red;
3596                pixel_out_rgb.green   <= rgb(24).gre;
3597                pixel_out_rgb.blue    <= rgb(24).blu;
3598            elsif((thr5.threshold25 = threshold)) then
3599                pixel_out_rgb.red     <= rgb(25).red;
3600                pixel_out_rgb.green   <= rgb(25).gre;
3601                pixel_out_rgb.blue    <= rgb(25).blu;
3602            elsif((thr5.threshold26 = threshold)) then
3603                pixel_out_rgb.red     <= rgb(26).red;
3604                pixel_out_rgb.green   <= rgb(26).gre;
3605                pixel_out_rgb.blue    <= rgb(26).blu;
3606            elsif((thr5.threshold27 = threshold)) then
3607                pixel_out_rgb.red     <= rgb(27).red;
3608                pixel_out_rgb.green   <= rgb(27).gre;
3609                pixel_out_rgb.blue    <= rgb(27).blu;
3610            elsif((thr5.threshold28 = threshold)) then
3611                pixel_out_rgb.red     <= rgb(28).red;
3612                pixel_out_rgb.green   <= rgb(28).gre;
3613                pixel_out_rgb.blue    <= rgb(28).blu;
3614            elsif((thr5.threshold29 = threshold)) then
3615                pixel_out_rgb.red     <= rgb(29).red;
3616                pixel_out_rgb.green   <= rgb(29).gre;
3617                pixel_out_rgb.blue    <= rgb(29).blu;
3618            elsif((thr5.threshold30 = threshold)) then
3619                pixel_out_rgb.red     <= rgb(30).red;
3620                pixel_out_rgb.green   <= rgb(30).gre;
3621                pixel_out_rgb.blue    <= rgb(30).blu;
3622            else
3623                pixel_out_rgb.red     <= (others => '1');
3624                pixel_out_rgb.green   <= (others => '1');
3625                pixel_out_rgb.blue    <= (others => '1');
3626    -- End of design block
3627            end if;
3628    -- End of design block
3629        end if;
3630    -- End of design block
3631    end process;
3632    pixel_out_rgb.valid <= rgbSyncValid(25);
3633    pixel_out_rgb.eol   <= rgbSyncEol(25);
3634    pixel_out_rgb.sof   <= rgbSyncSof(25);
3635    pixel_out_rgb.eof   <= rgbSyncEof(25);
3636    -- End of design block
3637    end architecture;
```